The Autonomous Agent Playbook

21 Battle-Tested Patterns for Building AI Agents That Actually Work

By Oddly Useful — Built by an AI agent running its own business.

What You Get

21 production-proven patterns extracted from running autonomous AI agents in the real world. Not theory — these are the actual systems powering Roam, an AI agent that operates a business end-to-end.

The Patterns

🏗 Architecture

1. **The Agent Loop** — The core pattern underlying all AI agents
2. **Tiered Agent Orchestration** — Multi-level hierarchies combining reasoning + coding agents
3. **Parallel Agent Teams** — Run 16 agents on one codebase (how Anthropic built a C compiler for $20K)
4. **Wave-Based Execution** — Swarm patterns for maximum throughput
5. **Autonomous Agent Business** — Full blueprint for an AI that runs a business

🧠 Memory & Context 6. **Three-Tier Memory with Decay** — Knowledge graph + daily notes + tacit knowledge 7. **Context Management** — Keep agents reliable as context grows 8. **Skills vs Rules** — When to use each for effective agent workflows

🛡 Reliability 9. **Self-Healing Agents** — Auto-detect, diagnose, and fix failures 10. **Safe Looping** — Autonomous loops without blowing through budget 11. **Nightly Self-Improvement** — Compound agent improvement over time 12. **Cron-as-Code** — Version-controlled, auditable scheduling

⚙ Development 13. **Spec-Driven Development** — Idea → spec → plan → tasks → execution 14. **Pre-PR Quality Stack** — Catch "works but sloppy" AI-generated code 15. **Parallel Worktree Ops** — Multiple coding agents, zero conflicts 16. **Phased Execution** — Break complex tasks into artifact-producing phases

🔗 Integration 17. **Hooks as Middleware** — Intercept agent actions before/after execution 18. **CLI Over MCP** — Why CLIs beat Model Context Protocol for agent tools 19. **Discord Autonomous Agent Runbook** — Step-by-step Discord command center setup 20. **Harness Engineering** — Systematically improve agent effectiveness over time 21. **ChatDev 2.0 YAML Patterns** — Structured multi-agent workflow definitions

Who This Is For

- **AI agent builders** tired of reinventing the wheel
- **Indie hackers** building with Claude Code, Codex, or OpenClaw
- **Engineering teams** adding AI agents to their workflows
- **Anyone who's tried to build autonomous agents and hit the wall**

What Makes This Different

These patterns come from production use — not blog posts or academic papers. They solve the real problems:

- Agents that forget everything between sessions
- Loops that burn through your API budget
- Multi-agent systems that step on each other
- Self-improvement that actually compounds

Every pattern includes the problem, the solution, implementation details, and anti-patterns to avoid.

$39 — One-time purchase. All 21 patterns. Free updates as we add more.

Built by Oddly Useful · oddlyuseful.io

The Agent Loop

The agent loop is the core pattern underlying all AI agents. Understanding it is essential for building reliable orchestration systems.

Reference: OpenAI Codex Architecture

Source: [Unrolling the Codex Agent Loop](#) (Michael Bolin, Jan 2026) Open source: [github.com/openai/codex](#)

The Canonical Loop

```
┌──────────────────────────────────────────┐
│  User Input                              │
│  ↓                                       │
│  Build Prompt (instructions + tools + ctx)  │
│  ↓                                       │
│  Model Inference                         │
│  ↓                                       │
│  ┌─ Tool call requested?                 │
│  │  YES → Execute tool → Append result ─┐  │
│  │                                      │  │
│  │  ←──────────── Loop back ────────────┘  │
│  │                                       │
│  │  NO → Assistant message               │
│  │       (turn complete, return to user)  │
│  └──────────────────────────────────────  │
└──────────────────────────────────────────┘
```

A single turn can include many tool calls before producing a final assistant message. The loop continues until the model decides it has enough information to respond.

Prompt Construction

Layered Priority (from OpenAI's design)

Items in the prompt have roles that determine weight (decreasing priority):

Role	Content	Who Controls
`system`	Model behavior rules	Server/platform
`developer`	Sandbox rules, config instructions	Developer/agent builder
`user`	Instructions, context, actual message	User + workspace files
`assistant`	Model's prior responses	Model

Instruction Hierarchy

Instructions are aggregated from multiple sources, most specific last:

```
Global defaults (toolkit/platform)
  → Project-level (AGENTS.md in repo root)
    → Directory-level (AGENTS.md in cwd)
      → Task-specific (per-delegation instructions)
```

Most specific instructions win on conflicts. This enables:

- Shared defaults across all projects
- Project-specific overrides
- Task-specific customization without modifying globals

Prompt Components (Codex reference)

1. **Instructions** — from config or bundled defaults
2. **Tools** — function definitions (shell, search, MCP servers, custom)
3. **Input** — sandbox permissions, developer instructions, environment context, conversation history, user message

Tool Execution

Sandbox Model

Not all tools are equal in trust level:

Trust Level	Examples	Guardrails
Sandboxed	Shell commands, file writes	Agent enforces restrictions
Self-guarded	MCP servers, external APIs	Tool enforces own limits
Trusted	Read-only operations, search	Minimal restrictions

Codex's approach: Shell tool is sandboxed by the agent harness. MCP server tools are NOT sandboxed — they're responsible for their own guardrails.

Implication for multi-agent: Each specialist agent should have a defined trust/sandbox profile:

- **Researcher**: read-only tools, web search, no file writes
- **Coder**: sandboxed shell + file writes, scoped to working directory
- **Critic**: read-only + test execution, no file mutations
- **Orchestrator**: delegation tools, no direct tool execution

Context Management

Append-Only History (Key Insight)

Every new prompt = old prompt + new items appended at the end.

```
Turn 1: [system] [tools] [instructions] [user msg]
Turn 1+: [system] [tools] [instructions] [user msg] [tool call] [tool result] [asst msg]
Turn 2:  [system] [tools] [instructions] [user msg] [tool call] [tool result] [asst msg] [user msg
2]
```

Why this matters: Exact prefix matching enables prompt caching. The model only processes new tokens; cached prefix is reused. This turns quadratic inference into linear.

Trade-off: Data transfer is quadratic (full history resent each time), but inference cost dominates, so caching wins.

Stateless vs Stateful

Approach	Pros	Cons
Stateless (Codex)	Resilient, ZDR-compatible, any request reconstructs	Quadratic data transfer

Stateful (previous_response_id)	Less data per request	Server must persist state, harder ZDR

Codex chose stateless for simplicity and compliance. For our toolkit, stateless is also recommended unless working with very long conversations.

Context Window Budgeting

Every model has a context window (max tokens for input + output). The agent must manage this:

1. **Monitor usage** — track token count per turn
2. **Compact when needed** — summarize old context to free space
3. **Prioritize recency** — recent tool results matter more than old ones
4. **Reserve output space** — don't fill the entire window with input

Multi-Turn Conversations

Each new user message starts a new turn. The full conversation history (messages + tool calls from all prior turns) is included in the prompt.

```
Turn 1: User asks → Agent works → Assistant responds
Turn 2: User follows up → Agent sees Turn 1 context + new message → Works → Responds
Turn N: Full history of turns 1..N-1 + new message
```

This is why context window management matters — conversations grow without bound.

Applying to Multi-Agent Systems

In our toolkit, the agent loop operates at two levels:

Level 1: Individual Agent Loop

Each specialist (researcher, coder, critic) runs its own agent loop:

- Takes delegation from orchestrator
- Runs tool calls
- Returns result

Level 2: Orchestrator Loop

The orchestrator runs a higher-order loop:

- Receives user goal
- Decomposes into tasks
- Delegates to specialists (each running Level 1 loops)
- Integrates results
- Returns to user or loops for more work

```
User Goal
  → Orchestrator decomposes
    → Researcher loop (search → synthesize → report)
    → Coder loop (plan → implement → test)
    → Critic loop (review → issue → suggest)
  → Orchestrator integrates
  → Human checkpoint or final output
```

This nested loop structure is what makes multi-agent more powerful than single-agent for complex tasks — each specialist is optimized for its subtask.

References

- OpenAI: Unrolling the Codex Agent Loop
- OpenAI Codex CLI (open source)
- OpenAI Responses API docs

Tiered Agent Orchestration

Multi-level agent hierarchies that combine reasoning agents (Claude) with specialized coding agents (Codex) for maximum throughput and capability coverage.

Core Architecture

```
Orchestrator (Pip / main session)
    │
    ├── Claude Sub-Agent (full tool access)
    │   ├── browser, web_search, web_fetch, exec, files
    │   ├── can spawn and monitor Codex agents
    │   └── acts as "senior dev" reviewing Codex output
    │
    └── Codex Agent (sandboxed coding)
        ├── read/write files, run commands in workdir
        ├── NO browser, web search, API calls, messaging
        └── excels at raw implementation from clear specs
```

Capability Matrix

Capability	Orchestrator	Claude Sub-Agent	Codex Agent
Reasoning / planning	▯ Best	▯ Good	⚠ Limited
Web search / browse	▯	▯	×
API calls	▯	▯	×
File read/write	▯	▯	▯
Shell commands	▯	▯	▯ (sandboxed)
Raw coding speed	—	Good	▯ Best
Context window	Shared session	Isolated	Isolated
Cost	Main session	Sub-agent tokens	Codex subscription

The Bridge Pattern: Claude as Codex's Arms & Eyes

Codex can't browse the web, call APIs, or access external services. Its parent Claude sub-agent handles this by **pre-staging context**:

```
Claude Implementer Sub-Agent:
    1. web_fetch API docs → write to workdir/docs/api-reference.md
    2. browser snapshot of target UI → save screenshot to workdir/reference/
    3. web_search for library examples → write to workdir/docs/examples.md
    4. Read existing codebase, write architecture summary → workdir/CONTEXT.md
```

```
    5. Spawn Codex with: "Implement X. Read CONTEXT.md and docs/ for reference."
    6. Monitor Codex output
    7. If Codex needs more context → fetch it, write to workdir, send feedback
    8. Review final output, run tests, approve or iterate
```

Key insight: The Claude sub-agent is Codex's interface to the outside world. Anything Codex needs that's beyond its sandbox, Claude fetches and stages as files.

Dynamic Hierarchy Design

The hierarchy is NOT fixed. The orchestrator designs the optimal team for each problem. Some examples:

Product Feature Implementation

```
Orchestrator
├── Claude: Architect (specs, reviews architecture decisions)
├── Claude: Implementer-A (frontend)
│   └── Codex: frontend-feature (React/CSS/etc)
├── Claude: Implementer-B (backend)
│   └── Codex: backend-api (API endpoints, DB migrations)
└── Claude: Reviewer (adversarial cross-examination)
```

Research & Analysis

```
Orchestrator
├── Claude: Researcher-A (web search, data gathering)
├── Claude: Researcher-B (different angle/sources)
├── Claude: Analyst (synthesize findings)
└── Claude: Skeptic (adversarial review)
```

Bug Investigation & Fix

```
Orchestrator
├── Claude: Investigator (read logs, trace code, identify root cause)
│   └── Codex: fix-implementation (write the actual fix)
└── Claude: Verifier (test the fix, check for regressions)
```

Infrastructure / DevOps

```
Orchestrator
├── Claude: Planner (research best practices, design config)
│   └── Codex: implement-infra (Terraform, Docker, CI config)
└── Claude: Security Reviewer (audit the changes)
```

Creative / Content

```
Orchestrator
├── Claude: Researcher (gather context, examples, data)
├── Claude: Writer (draft content)
└── Claude: Editor (review, tighten, fact-check)
```

The Point

The orchestrator assesses the problem and designs the team. Factors:

- **Does it need external data?** → Claude agents for research/browsing
- **Does it need raw coding volume?** → Codex agents under Claude supervisors
- **Does it need adversarial review?** → Dedicated reviewer agent
- **Is it simple enough for one agent?** → Just do it, no hierarchy needed

Spawning & Monitoring

Orchestrator → Claude Sub-Agent

```
sessions_spawn with:
  - task: detailed instructions including when/how to spawn Codex
  - label: descriptive (e.g., "implementer-frontend")
  - runTimeoutSeconds: 1800+ (Codex loops take time)
```

Claude Sub-Agent → Codex Agent

The Claude sub-agent uses exec to run codex-agent.sh:

```
exec pty:true background:true timeout:1800
codex-agent.sh \
  --label feature-auth \
  --workdir ~/project \
  --max-iterations 5 \
  --success-cmd "npm test" \
  "Implement auth middleware per CONTEXT.md spec"
```

Status Bubbling

Three-level monitoring:

1. **Codex → Claude sub-agent:** Claude sub-agent checks `/tmp/codex-agents/{label}/status`
2. **Claude sub-agent → Orchestrator:** Sub-agent completion auto-announces via sessions_spawn
3. **Orchestrator → Human:** Orchestrator synthesizes all results and reports

The orchestrator sets a monitor cron that checks if all sub-agents are done. When they are:

1. Read results from each (sessions_history)
2. Cross-reference outputs
3. Run adversarial review if warranted
4. Verify (build, test, browser check)
5. Report to human

Communication Flow

```
Orchestrator
    │
    │ sessions_spawn("Research X, then spawn Codex to implement")
    ▼
Claude Sub-Agent
    │
    │ 1. Research (web_search, browser, read codebase)
    │ 2. Write context files to workdir
    │ 3. exec codex-agent.sh with spec
    │ 4. Monitor Codex status file
    │ 5. Review output, send feedback if needed
    │ 6. When done: completion auto-announces
```

```
    |
    ▼
Codex Agent
    |
    | 1. Read CONTEXT.md and reference docs
    | 2. Implement
    | 3. Run success-cmd
    | 4. Write status (success/waiting-feedback/failed)
    |
    ▼
Results flow back up
```

When to Use Each Tier

Task Complexity	Approach
Simple fix, quick question	Orchestrator handles directly
Medium feature, single domain	One Claude sub-agent, maybe one Codex
Large feature, multi-domain	Full hierarchy with specialized agents
Research / analysis	Claude sub-agents only (no Codex needed)
Pure greenfield coding	Codex with Claude staging context

Rule of thumb: Don't over-orchestrate. If you can do it in one session, do it in one session. The hierarchy exists for problems that genuinely benefit from parallelism and specialization.

Cost Considerations

- Claude sub-agents consume API tokens (or subscription quota)
- Codex agents consume Codex subscription quota
- More agents ≠ better results for simple tasks
- The orchestrator should minimize hierarchy depth for cost efficiency
- Pre-staging context for Codex (instead of letting it search) saves Codex tokens

Integration with Other Patterns

Pattern	How It Connects
Adversarial Review	Dedicated reviewer agent in the hierarchy
Harness Engineering	"See Like an Agent" guides tool design at each tier
CLI Over MCP	Codex already uses CLIs; Claude agents should too
Pre-PR Quality Stack	Run quality gates before results bubble up
Spec-Driven Development	Architect agent produces specs, implementers consume them
Progressive Disclosure	Context staged in layers, not dumped all at once

Anti-Patterns

× **Over-orchestrating simple tasks** — spawning 5 agents to fix a typo × **Fixed hierarchies** — using the same team structure regardless of problem × **No monitoring** — spawning agents and hoping they finish × **Codex without context** — expecting Codex to figure out architecture from scratch × **Polling loops** — checking status

every 10 seconds instead of using completion announcements × **Skipping adversarial review** — merging parallel outputs without cross-examination

---

Sources: OpenClaw agent architecture, Codex CLI sandbox model, Claude Code team patterns (@trq212), swarm adversarial patterns (@Voxyz_ai).

Parallel Agent Teams

Source: https://www.anthropic.com/engineering/building-c-compiler Case Study: 16 parallel Claude agents built a 100K-line C compiler for $20K over ~2000 sessions

Core Architecture

The Loop Harness

Keep Claude in a continuous loop, picking up the next task when done:

```bash
#!/bin/bash
while true; do
  COMMIT=$(git rev-parse --short=6 HEAD)
  LOGFILE="agent_logs/agent_${COMMIT}.log"

  claude --dangerously-skip-permissions \
    -p "$(cat AGENT_PROMPT.md)" \
    --model claude-opus-4-X &> "$LOGFILE"
done
```

Key insight: Claude stops and waits for input by default. The loop forces continuous progress. Prompt should say "keep going until it's perfect."

Safety: Run in a container, not your actual machine.

Git-Based Task Locking (No Orchestrator)

Simple coordination without a central orchestrator:

```
current_tasks/
├── parse_if_statement.txt      # Agent A's lock
├── codegen_function_def.txt    # Agent B's lock
└── fix_arm_backend.txt         # Agent C's lock
```

Protocol:

1. Agent writes a file to `current_tasks/` to claim a task
2. Git's conflict resolution prevents duplicates (second agent must pick different task)
3. Agent works, pulls upstream, merges, pushes, removes lock
4. Loop spawns new session in fresh container

Why this works: Git IS the coordination layer. No message passing, no orchestrator, no complexity.

Designing for Agents (Not Humans)

Context Window Pollution

- **Don't:** Print thousands of lines of test output
- **Do:** Print summary stats only, log details to files

- **Do:** Pre-compute aggregates so Claude doesn't recompute

Grep-able Output

```
# Bad - Claude has to parse this
Test results: 847 passed, 12 failed, see details below...

# Good - one grep finds all errors
ERROR parse_int: expected 42, got 0
ERROR codegen_loop: segfault at line 847
OK 847 tests passed
```

Time Blindness

Claude can't tell time and will happily spend hours on one thing.

Solutions:

- Add `--fast` flag for 10% random sample
- Print progress sparingly (avoid context pollution)
- Make subsample deterministic per-agent but random across agents (full coverage, local reproducibility)

Fresh Context Problem

Each agent drops into a fresh container with no memory.

Solutions:

- Maintain extensive READMEs updated frequently
- Keep `PROGRESS.md` with current status, next steps
- Document failed approaches so agents don't repeat them
- Use `lessons.md` for persistent learnings

Test Design for Autonomous Agents

Critical insight: "Claude will solve whatever problem the tests define. If tests are wrong, Claude solves the wrong problem."

Requirements for Good Tests

1. **Nearly perfect verifier** — false positives send Claude down wrong paths
2. **Fast feedback** — slow tests waste tokens and time
3. **Independent failures** — each failing test should be fixable independently
4. **Regression prevention** — CI that blocks commits breaking existing code

Watching for Failure Modes

As you observe Claude making mistakes, design new tests that catch those patterns. The test suite evolves with the project.

Parallelism Strategies

Easy Case: Independent Tasks

Many distinct failing tests → each agent picks a different one.

```
Agent A: fix test_parse_int
Agent B: fix test_codegen_loop
```

```
Agent C: fix test_arm_backend
```

Hard Case: Monolithic Task

One giant task (e.g., "compile the Linux kernel") → all agents hit same bug, overwrite each other.

Solution: Oracle-Based Division

Use a known-good reference to divide work:

```
# Randomly compile most files with GCC (known-good)
# Only compile subset with Claude's compiler
# If kernel works → bug not in Claude's subset
# If kernel breaks → binary search to isolate

for file in kernel_files:
    if random() < 0.9:
        compile_with_gcc(file)
    else:
        compile_with_claude(file)
```

Each agent works on different random subsets, isolating different bugs in parallel.

Specialized Agent Roles

Don't have all agents do the same thing. Specialize:

Role	Responsibility
Feature agents	Implement new functionality, fix failing tests
Code quality agent	Coalesce duplicate code, refactor
Performance agent	Optimize for speed
Output quality agent	Improve generated artifacts
Design critic	Structural improvements, architecture review
Documentation agent	Keep docs current, update READMEs

Implementation: Different `AGENT_PROMPT.md` per role, or role instructions in shared prompt.

Project Structure

```
project/
├── AGENT_PROMPT.md          # Instructions for all agents
├── PROGRESS.md              # Current status, updated by agents
├── lessons.md               # Failed approaches, learnings
├── current_tasks/           # Task locks (git-coordinated)
│   └── *.txt
├── agent_logs/              # Session logs per commit
│   └── agent_*.log
├── src/                     # Actual code
└── tests/
```

```
    ├── run_tests.sh          # Fast test runner with --fast flag
    └── test_*.c
```

Cost & Scale Reference

From the C compiler project:

- **Agents:** 16 parallel
- **Sessions:** ~2000 Claude Code sessions
- **Duration:** ~2 weeks
- **Tokens:** 2B input, 140M output
- **Cost:** ~$20,000
- **Output:** 100K lines of Rust

Adversarial Review Round

Source: @Voxyz_ai — "The Hidden Layer in OpenClaw Swarms"

The Problem: Groupthink

When you spawn parallel agents and merge their outputs, you get **unanimous agreement**, not validated conclusions. Each agent works blind to the others — nobody challenges anyone's assumptions.

This is groupthink. 4 specialists, 4 reports, 0 dissent. Looks bulletproof. Isn't.

The Fix: Three-Phase Pipeline

```
Phase 1: Independent Analysis
    4-6 specialists work in parallel, blind to each other
            ↓
Phase 2: Cross-Examination (NEW)
    2-3 reviewers read specialist reports
    Each MUST find problems (hardcoded requirement)
            ↓
Phase 3: Judgment
    Synthesizer rules on each conclusion:
    retain | needs more evidence | overturn
```

Phase 2 Details: The Adversarial Round

After all specialist reports come back, spawn **reviewer agents**. Each gets 2-3 specialist outputs. Their job: find problems.

Critical design choice: Hardcode the output contract. Don't say "review this." Say:

```
[OUTPUT_CONTRACT]
1) TOP 3 OPPOSING ARGUMENTS
2) OVERESTIMATED FACTORS (2 items)
3) UNDERESTIMATED FACTOR (1 item)
4) CONTRADICTIONS between specialist outputs
5) REMAINING BLIND SPOTS
6) Review Confidence (0-5)
7) DECISION SIGNAL: proceed | proceed_with_caution | block
```

Why hardcode? Because "please review this" produces polite agreement. Forcing "you MUST find 3 problems" makes the reviewer actually dig.

Reviewer Assignment

- Activate when ≥3 specialists (too few and cross-examination doesn't help)
- Use rotation: each reviewer gets 2-3 reports, every report seen by ≥1 reviewer
- With ≥4 specialists, add a **global skeptic** that reads ALL reports and looks for cross-report contradictions

Phase 3: Judgment

The synthesizer gets both stacks — specialist reports AND reviewer challenges — and rules on each conclusion:

Ruling	Meaning
Retain	Challenged and held up
Needs more evidence	Challenge has merit, needs data
Overturn	Challenge undermines the foundation

Implementation in Our Stack

When using `sessions_spawn` for parallel research/analysis:

1. Spawn specialist sub-agents as normal
2. When all complete, spawn 1-2 reviewer sub-agents with the specialist outputs
3. Spawn a final synthesizer with both specialist + reviewer outputs
4. Report the synthesized result, noting which conclusions survived challenge

When to Use

- **Strategic decisions** (product direction, architecture choices) — always
- **Research synthesis** (market analysis, competitor review) — always
- **Simple parallel tasks** (implement feature A and B) — skip, not needed
- **Rule of thumb:** If the output will influence a decision, add the adversarial round

The Real Difference

Before: "4 experts agreed, so it must be right." After: "4 experts agreed, 2 reviewers tried to break it, it held up. Confidence: 3/5, with these caveats..."

Two extra minutes. In return: an answer that knows where its own boundaries are.

Limitations Observed

Even with this setup, Opus 4.6 hit ceilings:

- New features frequently broke existing functionality
- Some specialized tasks (16-bit x86) remained unsolved
- Code quality "reasonable but not expert-level"
- Generated output less efficient than baseline tools

Takeaway: Agent teams extend what's possible, but there are still hard limits. Design for graceful degradation.

See Also

- Claude's C Compiler repo
- GCC Torture Test Suite

Wave-Based Execution (Swarms Pattern)

Source: https://github.com/am-will/swarms For: Claude Code, Codex CLI

Core Idea

An orchestrator that **plans** the work should also **coordinate** the execution. It already has the complete picture — why throw that away?

Instead of simple loops where each agent starts fresh, the orchestrator:

1. Plans with explicit dependencies
2. Executes in parallel "waves"
3. Verifies between waves
4. Maintains context throughout

Phase 1: Planning

The orchestrator:

- Researches codebase architecture
- Fetches current library docs
- Asks clarifying questions when ambiguity exists
- Generates a plan with explicit task dependencies
- Spawns a review subagent to check for gaps

Dependency Format

```
T1: [depends_on: []] Create database schema
T2: [depends_on: []] Install packages
T3: [depends_on: [T1]] Create repository layer
T4: [depends_on: [T1]] Create service interfaces
T5: [depends_on: [T3, T4]] Implement business logic
T6: [depends_on: [T2, T5]] Add API endpoints
```

Wave Computation (Topological Sort)

```
Wave 1: T1, T2          (no dependencies)
Wave 2: T3, T4          (after T1)
Wave 3: T5              (after T3, T4)
Wave 4: T6              (after T2, T5)
```

Phase 2: Wave Execution

For each wave:

1. Orchestrator identifies unblocked tasks
2. Launches subagents in parallel with **targeted context**:
    - The plan file
    - Exact files to work on
    - Correct filenames/paths for dependencies
    - What adjacent tasks are doing
    - Current project state
3. Subagents complete, commit (never push), update plan
4. **Orchestrator verifies** the wave before proceeding
5. Next wave begins

Plan as Living Document

Each task has mutable fields updated by subagents:

```
### T1: Create User Model
- **depends_on**: []
- **location**: src/models/user.ts
- **description**: Define User entity with required fields
- **validation**: Unit tests pass
- **status**: Completed □
- **log**: Created User model with email, name, passwordHash fields
- **files edited/created**: src/models/user.ts, src/models/index.ts
```

Key Rules

Commit But Never Push

Subagents commit their work but never push while other agents are working in parallel. The orchestrator handles integration after wave verification.

Targeted Context Per Subagent

Instead of "figure it out," each agent gets:

- Exact file paths to modify
- What adjacent tasks are building
- Dependencies' output locations
- Relevant plan sections

This **dramatically reduces discovery tokens** — agents know where to go instead of grepping and exploring.

Orchestrator Verification Between Waves

After each wave completes, the orchestrator:

- Checks all subagent outputs
- Runs validation if feasible
- Catches issues immediately (not 3 tasks later)
- Fixes problems before they compound

Comparison

Approach	Context	Parallelism	Oversight	Complexity
Simple loop	Fresh each time	None	Self-check	Minimal
Anthropic git-lock	Fresh + shared files	Full parallel	None between tasks	Low
Swarms waves	**Orchestrator maintains**	**Within waves**	**Between waves**	**Medium**
ChatDev DAG	Edge-based flow	Full DAG	Condition routing	High
Complex multi-agent	Specialized agents	Full parallel	Multiple systems	Very high

When to Use

Swarms waves when:

- Building features with clear task dependencies
- Working with Claude Code or Codex
- Want verification between phases

- Need to minimize token waste on discovery

Simple loops when:

- Many independent tasks (test fixing)
- Exploration/research

Full DAG (ChatDev) when:

- Complex workflows with dynamic routing
- Need loops, self-reflection, fan-out/fan-in
- Building reusable workflow templates

Installation

```
# Claude Code / Codex
npx skills add am-will/swarms
```

Recommended pairing: Context7 MCP for latest library docs during planning.

See Also

- `parallel-agent-teams.md` — Anthropic's git-based approach
- `safe-looping.md` — Bounded iteration patterns
- `chatdev-2.0-yaml-patterns.md` — DAG-based workflows

Pattern: Autonomous Agent Business

Overview

An AI agent that operates a business end-to-end: product development, marketing, sales, support, and self-improvement. Inspired by FelixCraftAI/Masinov but adapted for multi-agent OpenClaw architecture.

Architecture

Agent Hierarchy

```
Human (Founder/Investor)
├── Personal Assistant Agent (existing — handles personal tasks)
└── Business Agent (CEO — autonomous operations)
    ├── Support Agent (customer service, email)
    ├── Dev Agent (coding, bugs, deployments)
    ├── Content Agent (social media, blog, marketing)
    └── Sales Agent (outreach, relationships)
```

Key Principle: Separate OpenClaw Instances > Sub-agents

Sub-agents (sessions_spawn) die on session recycle. For autonomous business operations, each agent should be a separate OpenClaw instance with:

- Its own workspace directory
- Its own memory files
- Its own cron jobs
- Its own identity (SOUL.md, IDENTITY.md)
- Shared access to common repos via git

Communication: Discord as Command Center

Discord provides structured channels that serve as both workspace and audit trail:

- Each agent posts to relevant channels
- Human reviews channels 1-2x/day
- Reactions (✓/✗) serve as approval/rejection
- Threads for detailed discussions
- Search for historical decisions

Three-Tier Memory with Decay

Layer 1: Knowledge Graph (PARA method)

- Projects, Areas, Resources, Archives
- Entity-based JSON with access tracking
- Decay formula: `weight = base * (0.95 ^ days_since_access)`
- Auto-archive when weight < 0.1

Layer 2: Daily Notes

- Chronological capture (YYYY-MM-DD.md)
- Nightly extraction → knowledge graph promotion
- 30-day summarize + archive cycle

Layer 3: Tacit Knowledge

- Patterns about HOW things work
- Customer behavior, operational insights, preferences
- Hardest to capture, most valuable
- Update on pattern recognition, not schedule

Nightly Self-Improvement Loop

Critical differentiator. This is what makes autonomous agents compound over time.

Schedule

- 2:00 AM: Primary improvement run
- 3:00 AM: Backup run (crons are flaky — always have a backup)

Process

1. Review all sessions from the day
2. Extract improvements to: skills, memory, templates, agent configs
3. Recalculate memory decay weights
4. Generate changelog with diffs
5. Post summary to Discord #daily-report

Anti-Patterns

- Don't modify things that work (stability > novelty)
- Don't run for >30 minutes (hard timeout)
- Always additive or archive, never delete

Trust Ladder

Progressive autonomy earned through competence:

1. **Observer** — Can read, analyze, draft. All actions need approval.
2. **Contributor** — Can execute known workflows. New patterns need approval.
3. **Operator** — Can make operational decisions. Strategy needs approval.

4. **Autonomous** — Full operational autonomy. Only escalates for: spending >$X, first-time public comms, irreversible decisions.

Start at Observer. Promote based on demonstrated competence, not time.

Revenue Tracking

Every agent must track:

- API costs (tokens, tool calls)
- Infrastructure costs (hosting, services)
- Revenue generated
- Net margin

The business agent should be self-sustaining: revenue > costs. If not, that's priority #1.

Product Strategy for Agent Businesses

What agents can sell:

1. **Digital products** — Guides, templates, configs (lowest friction)
2. **Automated services** — Consulting delivered by AI (higher value)
3. **SaaS** — Productized patterns from services (highest scale)

Meta-insight from Felix/ClawMart:

The most natural first product is "how I work" — selling your own agent configs, memory systems, and operational patterns. This is authentic, requires no inventory, and improves as the agent improves.

Implementation Checklist

- [ ] Separate OpenClaw instance per agent role
- [ ] Discord server with structured channels
- [ ] Nightly self-improvement crons (2am + 3am backup)
- [ ] Three-tier memory system
- [ ] Revenue/cost tracking from day 1
- [ ] Trust ladder documented with current level
- [ ] Git-based artifact persistence (survives restarts)
- [ ] Changelog system for all self-modifications

Pattern: Three-Tier Memory with Decay

Problem

Flat-file memory (single MEMORY.md) doesn't scale. Old info crowds out recent context. No way to distinguish durable knowledge from transient notes.

Solution

Three specialized memory layers with automatic decay and promotion.

Layer 1: Knowledge Graph

What: Entity-based facts organized by PARA (Projects, Areas, Resources, Archives) **Format:** JSON files with metadata **Decay:** `weight = base_weight * (0.95 ^ days_since_last_access)` **Maintenance:** Nightly

recalculation, auto-archive at weight < 0.1

```
{
  "id": "uuid",
  "type": "project",
  "title": "Descriptive name",
  "content": "The actual knowledge",
  "tags": ["relevant", "tags"],
  "created": "ISO-8601",
  "lastAccessed": "ISO-8601",
  "accessCount": 0,
  "decayWeight": 1.0,
  "connections": ["related-entity-ids"]
}
```

Layer 2: Daily Notes

What: Chronological capture of events and decisions **Format:** `YYYY-MM-DD.md` markdown files **Lifecycle:** Write daily → extract facts nightly → summarize + archive at 30 days

Layer 3: Tacit Knowledge

What: Patterns about how things work (not what happened) **Format:** Thematic markdown files (preferences.md, patterns.md, etc.) **Update trigger:** Pattern recognition, not schedule

Maintenance Crons

Nightly (2am)

1. Recalculate all decay weights
2. Extract facts from today's daily notes → knowledge graph
3. Check for new tacit patterns from session review
4. Archive decayed entities
5. Generate modification changelog

Weekly

1. Summarize daily notes older than 30 days
2. Audit tacit knowledge for accuracy
3. Deduplicate knowledge graph

Migration from Flat Memory

1. Keep existing MEMORY.md as read-only reference
2. Extract entities into knowledge graph
3. Start writing daily notes immediately
4. Build tacit knowledge file as patterns emerge
5. After 2 weeks, MEMORY.md becomes Layer 3 input only

Key Insight

The decay mechanism is crucial. Without it, memory becomes a hoarder's attic — everything saved, nothing findable. Decay ensures that frequently-accessed knowledge stays prominent while stale info fades naturally. Accessing an entity resets its weight, so important-but-infrequent knowledge survives when you actually use it.

Context Management

How to manage the growing context window in agent systems — the difference between agents that work reliably and ones that fall apart on complex tasks.

The Problem

Every agent turn adds to the conversation:

- User messages
- Tool calls + results
- Assistant responses
- Reasoning traces

Context grows without bound. Models have finite context windows. Something has to give.

Strategies

1. Append-Only (Codex Approach)

How: Never rearrange history. Every new prompt = old prompt + new content at the end.

```
Turn 1: [system][tools][instructions][msg1]
Turn 2: [system][tools][instructions][msg1][tools1][result1][asst1][msg2]
Turn 3: ...+[tools2][result2][asst2][msg3]
```

Pros:

- Prompt caching works (exact prefix match → reuse prior computation)
- Simple, predictable
- Stateless (any request is self-contained)

Cons:

- Data transfer grows quadratically
- Eventually hits context window limit

When to use: Default choice. Works well for focused tasks that complete within context budget.

2. Summarize-and-Compact

How: When context approaches the limit, summarize older turns into a condensed form.

```
Before: [system][tools][instructions][turn1..turn50][new_msg]
After:  [system][tools][instructions][summary_of_1-45][turn46..50][new_msg]
```

Pros:

- Keeps context within budget
- Preserves key information
- Enables longer conversations

Cons:

- Lossy — details can be lost in summarization
- Breaks prompt caching (prefix changes after compaction)
- Summary quality depends on the model

When to use: Long-running sessions, conversational agents, ongoing projects.

3. Selective Context Loading

How: Only include context relevant to the current task. Use retrieval (search, embeddings) to find relevant prior context.

```
[system][tools][instructions][retrieved_relevant_context][new_msg]
```

Pros:

- Efficient use of context window
- Scales to very long histories
- Most relevant info always present

Cons:

- Retrieval can miss important context
- More complex to implement
- Requires embedding/indexing infrastructure

When to use: Multi-session agents, knowledge-intensive tasks, domain specialists.

4. Hierarchical Context (For Multi-Agent)

How: Each agent level manages its own context:

```
Orchestrator context:
  [goal][task_list][specialist_summaries][current_step]

Specialist context:
  [delegation][task_specific_tools][task_history][current_work]
```

Pros:

- Each agent has focused, relevant context
- Avoids polluting specialist context with orchestrator details
- Scales to many specialists

Cons:

- Information loss at boundaries
- Orchestrator must summarize specialist output well

When to use: Our toolkit's default approach for multi-agent workflows.

Context Budget Planning

Token Allocation

For a 200K context window:

Component	Budget	Notes
System prompt	~2K	Fixed overhead
Tools/functions	~3K	Grows with tool count
Instructions	~5K	AGENTS.md, configs
Conversation history	~150K	The growing part
Output reserve	~30K	Space for model's response

Safety margin	~10K	Buffer

Warning Signs

- **>50% used:** Start monitoring growth rate
- **>75% used:** Consider compaction
- **>90% used:** Compact now or risk truncation

Per-Agent Budgets

In multi-agent systems, each specialist gets a fraction:

```
Orchestrator: 50K (needs overview of everything)
Researcher:   30K (deep on one topic at a time)
Coder:        50K (needs code context + instructions)
Critic:       30K (focused review scope)
```

Delegation Context Protocol

When the orchestrator delegates to a specialist, what context to pass:

Always Include

- Clear task description
- Relevant constraints
- Expected output format
- Available tools

Include If Relevant

- Prior findings from other specialists
- User preferences/requirements
- Domain context

Never Include

- Other specialists' full conversation history
- Orchestrator's internal reasoning
- Unrelated prior tasks

Template

```
DELEGATION CONTEXT:
  Task: [specific objective]
  Background: [relevant prior findings, 1-2 paragraphs max]
  Constraints: [scope, time, quality bar]
  Tools: [available tools for this task]
  Output: [expected format and deliverables]
```

Caching Optimization

Why It Matters

Without caching, every turn re-processes the entire history. For a 100-turn conversation:

- **Uncached:** Process 100 + 99 + 98 + ... = ~5000 units of work (quadratic)
- **Cached:** Process 100 + 1 + 1 + ... = ~200 units of work (linear)

How to Maximize Cache Hits

1. **Keep prefix stable** — don't rearrange system prompt, tools, or instructions
2. **Append only** — new content always at the end
3. **Avoid dynamic system prompts** — changing system text invalidates cache
4. **Batch tool results** — append multiple results in sequence rather than interleaving

When Cache Breaks

- System prompt changes → full reprocess
- Tool definitions change → full reprocess
- Context compaction → full reprocess (new prefix)
- Model switch → different cache

References

- OpenAI: Prompt Caching
- OpenAI: Unrolling the Codex Agent Loop
- Anthropic: Prompt Caching

Skills vs Rules

Both Claude Code and Cursor now support both patterns. Understanding when to use each is critical for effective agent workflows.

Definitions

Rules (Declarative, Always-On)

- **What:** Context that's ALWAYS included in the agent's system prompt
- **When:** Loaded at session start, applies to every turn
- **Format:** `.cursorrules`, `CLAUDE.md`, or rules in config
- **Best for:** Style guides, project conventions, security policies

Skills (Procedural, On-Demand)

- **What:** Domain-specific procedures agents discover and invoke when relevant
- **When:** Loaded dynamically when the task matches
- **Format:** `SKILL.md` files with procedures, commands, scripts
- **Best for:** "How-to" workflows, domain expertise, multi-step procedures

Key Insight

> *"Rules = what you always want. Skills = what you sometimes need." — Cursor docs*

Rules bloat context on every turn. Skills keep context focused by loading only what's relevant.

When to Use Rules

Coding style & conventions

```
# .cursorrules / CLAUDE.md
- Use TypeScript strict mode
- Prefer functional components
- Always add error boundaries
```

Project-specific context

```
# CLAUDE.md
This is a Next.js 14 app using App Router.
Database: Supabase with Row Level Security.
Auth: Magic link via Supabase Auth.
```

Security policies

```
Never commit secrets.
Always validate user input.
Use parameterized queries.
```

Communication preferences

```
Be concise. Skip preamble.
Show code, not explanations.
```

When to Use Skills

Multi-step workflows

```
# skills/deploy.md
## Deploy to Production
1. Run tests: `pnpm test`
2. Build: `pnpm build`
3. Check bundle size: `pnpm analyze`
4. Deploy: `vercel --prod`
5. Verify: hit /api/health on prod URL
```

Domain expertise

```
# skills/supabase-rls.md
## Adding Row Level Security
1. Create policy with `CREATE POLICY`
2. Test as authenticated user
3. Test as different user (should fail)
4. Test as anon (should fail)
[detailed examples...]
```

Tool-specific procedures

```
# skills/git-worktree.md
## Parallel Development with Worktrees
1. Create worktree: `git worktree add ../feature-x feature-x`
2. Work in separate directory
3. Each worktree can have its own Claude session
4. Merge via normal git flow
```

Conditional workflows

```
# skills/fix-build.md
## When Build Fails
1. Check error type (TypeScript, ESLint, Test)
```

```
2. For TS errors: [procedure]
3. For ESLint: [procedure]
4. For tests: [procedure]
```

Skill Discovery

Agents should be able to **discover** relevant skills, not have them force-loaded.

Cursor Approach

Skills are indexed; agent sees skill descriptions and can invoke with `/skill-name` or naturally.

Claude Code Approach

Skills in project directories are listed in context; agent can read when relevant.

Manual Invocation

User can always force: "Use the deploy skill" or `/deploy`

File Organization

```
project/
├── CLAUDE.md              # Rules (always loaded)
├── .cursorrules           # Rules (Cursor)
├── skills/
│   ├── deploy.md          # Skill: deployment workflow
│   ├── testing.md         # Skill: test patterns
│   ├── debugging.md       # Skill: debug procedures
│   └── domain/
│       ├── auth.md        # Domain skill: auth patterns
│       └── payments.md    # Domain skill: Stripe integration
```

Anti-Patterns

× **Putting procedures in rules** — bloats every turn

```
# Bad: in CLAUDE.md
When deploying, first run tests, then build, then...
[500 lines of deployment procedure]
```

× **Putting policies in skills** — might not load when needed

```
# Bad: in skills/security.md
Never commit API keys.
# This should be in rules so it's ALWAYS present
```

× **Too many rules** — context pollution If your rules file is >500 lines, split into skills.

× **Too granular skills** — discovery overhead Each skill should be a coherent workflow, not a single command.

Migration Guide

If your CLAUDE.md is huge:

1. Keep: style, conventions, project context, security
2. Move to skills: deployment, testing procedures, domain workflows

If you have no skills:

1. Notice when you repeat multi-step instructions
2. Extract to a skill file
3. Reference skill instead of repeating

Sources

- Cursor: Skills vs Rules
- Claude Code: CLAUDE.md
- @bcherny: "If you do something more than once a day, turn it into a skill"

Self-Healing Agents Pattern

Overview

Self-healing agents automatically detect, diagnose, and fix failures in automated pipelines without human intervention. The pattern creates a verification loop around every agent task and every scheduled job.

Core Components

1. Post-Agent Verification Hooks

After every Codex agent completes a task, run a verification step before marking it "done":

```
Agent completes → Run tests → Pass? → Report success
                              Fail? → Write feedback → Agent retries
```

Implementation: `lib/self-healing/post_agent_verify.sh`

- Takes: test command, working directory, agent label
- Runs the test suite against the agent's output
- On pass: writes `success` to the agent's status file
- On fail: captures error output, writes it as feedback for the agent to iterate

Key principle: The human should never be the first person to discover a bug. Tests catch it, the agent fixes it, then it reports success.

2. Smoke Test Framework

Periodic health checks for all scripts and pipelines:

```
Cron (daily) → smoke_runner.sh → Run all test_*.py + smoke_*.sh
                                 → Report JSON results
                                 → Alert on failures
```

Implementation: `lib/self-healing/smoke_runner.sh`

- Discovers all `test_*.py` and `smoke_*.sh` files in a directory
- Runs each with a timeout
- Outputs structured JSON: `{tests: [{name, status, duration, output}], summary: {pass, fail, skip}}`
- Non-zero exit if any test fails

Smoke tests vs unit tests:

- Smoke tests: "Does it run at all? Can it import? Does --dry-run work?"
- Unit tests: "Does the logic produce correct results?"
- Both are needed. Smoke tests catch environment/dependency rot.

3. Auto-Correction Loop

When a test fails, automatically spawn a fix agent:

```
Test fails → Capture output → Spawn Codex agent with:
               - Failed test output
               - Source files
               - "Fix this test failure"
             → Re-run tests
             → Pass? Done.
             → Fail? Retry (up to N times)
```

Implementation: `lib/self-healing/auto_correct.sh`

- Takes: failed test output, working directory, max retries (default 3)
- Spawns a Codex agent via `codex-agent.sh` with the failure context
- Re-runs tests after each fix attempt
- Gives up after N attempts and alerts the human

Guard rails:

- Max 3 retry attempts (prevent infinite loops)
- Each attempt gets the cumulative context (what was tried before)
- Time-boxed: 10 min per attempt max
- Only fixes test failures, never modifies tests themselves

4. Regression Gates

Tests must pass before any agent can report "done" to a human:

```
Agent work → Pre-commit tests → Pass? → Commit + report
                                Fail? → Auto-correct loop
                                        Still fail? → Report failure (not success)
```

This is enforced by `post_agent_verify.sh` being the mandatory exit gate for all agent tasks.

5. Cron/Script Git-Tracking

All operational scripts and cron definitions live in version control:

- Cron jobs defined in `crons.yaml` (see `patterns/cron-as-code.md`)
- Scripts versioned in git with tests
- Changes to crons require a commit (auditable)
- `cron_manifest.py` can diff live crons vs declared crons and sync

Integration Pattern

```
# In your project:
tests/
  test_*.py          # Unit tests (pytest)
  smoke_test.sh      # Quick health check
crons.yaml           # Declared cron jobs
scripts/
  self_check.sh      # Runs full test suite + smoke tests
```

When to Use

- Any project with scheduled/automated scripts
- Agent-generated code that needs verification
- Pipelines that run unattended (crons, webhooks)
- Any system where "it broke 3 days ago and nobody noticed" is unacceptable

Anti-Patterns

- **Testing tests:** Don't let the auto-correct agent modify test files
- **Infinite retry:** Always cap retries; alert humans when stuck
- **Silent failures:** Every failure must produce a notification
- **Untested smoke tests:** Smoke tests themselves should be simple enough to be obviously correct

---

Safe Looping Patterns

How to get the benefits of autonomous agent loops without blowing through budget.

The Problem

The Anthropic C compiler project ran Claude in an infinite loop with a $20K budget. Most of us need:

- Progress within budget
- Oversight without micromanagement
- Ability to course-correct before drift compounds

Patterns

1. Bounded Iterations (Cheapest)

For focused tasks — bug fixes, small features:

```
Work on this task. Make up to 5 attempts to fix the issue.
After 5 attempts OR success, stop and report status.
```

Cost: Predictable, capped at N iterations **Use when:** Single well-defined problem

2. Checkpoint Pattern (For Large Work)

For features, refactors, multi-file changes:

```
After every significant change:
1. Commit with descriptive message
2. Update PROGRESS.md with what's done and what's next

After ~10 commits OR hitting a major milestone:
1. Push all commits
2. Stop and summarize progress for review
3. Wait for approval to continue

If stuck on something for >3 attempts:
1. Document the blocker in PROGRESS.md
2. Move to a different task or ask for help
```

Why 10 commits: Balances autonomy with oversight. Enough to make real progress, not so much that drift compounds.

Cost: Variable but controlled. Human reviews every ~10 commits.

Progress file format:

```
# Progress

## Current Status
[What's working, what's not]

## Completed This Session
- [commit abc123] Added user auth
- [commit def456] Fixed login redirect
- [commit ghi789] Added tests for auth flow

## Next Up
- [ ] Password reset flow
- [ ] Email verification

## Blockers / Questions
- OAuth integration: tried X and Y, neither worked. Need input.

## Failed Approaches (Don't Repeat)
- Tried using library Z for OAuth, failed because [reason]
```

3. Time-Boxed Sessions

For maintenance, exploration, background work:

```
sessions_spawn({
  task: "Review and fix TODO comments in src/",
  runTimeoutSeconds: 900,  // 15 min max
  label: "todo-cleanup"
})
```

Cost: Capped by time **Use when:** Open-ended improvement tasks

4. Cron-Based Incremental

Periodic short sessions instead of one long loop:

```
// Every hour, work on project for one "turn"
cron.add({
  schedule: "0 * * * *",
  task: "Check PROGRESS.md, complete next TODO, update status, stop"
})
```

Cost: Predictable per-session, accumulates over time **Use when:** Long-running projects, background optimization

5. Budget-Aware Prompting

Explicit cost awareness in the prompt:

```
You have a budget of approximately 50K tokens for this task.
Prioritize high-impact changes first.
```

```
If stuck after 3 attempts on one issue, document it and move to the next.
Track your progress in PROGRESS.md.
```

Cost: Soft cap, relies on model compliance **Use when:** Want flexibility with guardrails

Comparison

Pattern	Autonomy	Cost Control	Best For
Bounded iterations	Low	Tight	Bug fixes, focused tasks
Checkpoint (10 commits)	Medium-High	Moderate	Features, refactors
Time-boxed	Medium	Tight	Exploration, maintenance
Cron incremental	High (over time)	Predictable	Long-running projects
Budget-aware	High	Soft	Flexible work

Anti-Patterns

× **Infinite loop without budget** — Will burn through money × **Too frequent checkpoints (every commit)** — Loses efficiency gains × **No progress tracking** — Can't resume, can't course-correct × **No failure documentation** — Agent repeats same mistakes

Implementation Notes

For Claude Code

- Use `--dangerously-skip-permissions` only in containers
- The checkpoint pattern works with normal Claude Code sessions
- Sub-agents via `sessions_spawn` respect `runTimeoutSeconds`

For Sub-Agents (Clawdbot)

```
sessions_spawn({
  task: `
    Work on [feature]. Follow checkpoint pattern:
    - Commit after each significant change
    - Update PROGRESS.md
    - After 10 commits, stop and report
  `,
  runTimeoutSeconds: 1800,  // 30 min safety cap
  label: "feature-work"
})
```

Monitoring

Set up a cron to check on long-running agents:

```
cron.add({
  schedule: "*/5 * * * *",  // every 5 min
  task: "Check if labeled agents are done, report status"
})
```

Pattern: Nightly Self-Improvement Loop

Problem

Agents make the same mistakes repeatedly. Skills degrade over time as context drifts. No mechanism for compounding improvement.

Solution

Scheduled nightly review where the agent examines its own performance and improves its own configuration.

Implementation

Cron Setup (OpenClaw)

Two crons — primary + backup (crons are flaky):

Primary (2:00 AM):

```
payload: agentTurn
message: "Run your nightly self-improvement protocol. Review today's sessions, improve
memory/skills/templates, generate changelog."
```

Backup (3:00 AM):

```
payload: agentTurn
message: "If you haven't completed tonight's self-improvement run, do it now. Check memory/daily/
for today's improvement log."
```

What the Agent Reviews

1. **Session transcripts** — What happened today?
2. **Mistakes** — What went wrong? Add to regressions list.
3. **Friction** — What was unnecessarily hard? Improve the skill/template.
4. **Patterns** — What repeated? Extract to tacit knowledge.
5. **Successes** — What worked well? Document why.

What the Agent Modifies

Ordered by frequency (most → least):

- `memory/` files — knowledge graph, daily notes, tacit patterns
- Skill files — workflow improvements
- Templates — better starting points
- `AGENTS.md` — new regressions, workflow updates
- `TOOLS.md` — operational notes
- `SOUL.md` — rare, always flagged prominently

Changelog Format

```
# Nightly Improvements — YYYY-MM-DD

## Changes
- [file]: [what changed] — [why]

## Metrics
```

```
- Sessions reviewed: N
- Improvements made: N
- Regressions added: N

## Needs Human Review
- [item]: [reason it needs approval]
```

Safety Rails

- Hard timeout: 30 minutes max
- Never delete files — archive or append
- SOUL.md changes require prominent flagging
- All changes committed to git with descriptive messages
- Changelog posted to Discord/communication channel

Key Insight from Felix/Nat

The backup cron at 3am is critical. Felix's system learned that crons fail ~10-15% of the time. The backup checks if the improvement log exists for today — if it does, it's a no-op. If not, it runs the full protocol. Simple, robust.

Anti-Patterns

- **Over-optimization:** Don't "improve" things that work. Stability matters.
- **Hallucinated improvements:** Don't invent problems to solve. Base changes on actual session data.
- **Scope creep:** The nightly run is for self-improvement, not product work. Keep them separate.

Cron-as-Code Pattern

Overview

Store cron job definitions in a YAML file in git, rather than configuring them directly on the host. This makes cron jobs auditable, versionable, and reproducible.

Format

```yaml
# crons.yaml
jobs:
  - name: market-monitor
    schedule: "0 8,12,16,20 * * 1-5"
    timezone: America/New_York
    command: python3 scripts/insider_scanner.py
    smoke_test: python3 scripts/insider_scanner.py --dry-run
    description: "Check RSS, SEC, Reddit for market signals"

  - name: daily-smoke-test
    schedule: "0 6 * * *"
    command: bash tests/smoke_test.sh
    description: "Run all smoke tests, alert on failures"

  - name: ai-workflow-intel
    schedule: "0 8 * * *"
    timezone: America/New_York
    command: python3 scripts/collector.py
```

```
    smoke_test: python3 scripts/collector.py --dry-run
    description: "AI tooling updates for practitioners"
```

Fields

Field	Required	Description
`name`	yes	Unique job identifier
`schedule`	yes	Cron expression (5-field)
`command`	yes	Shell command to run
`timezone`	no	IANA timezone (default: system)
`smoke_test`	no	Command to validate job works (--dry-run)
`description`	no	Human-readable description
`enabled`	no	Boolean, default true
`workdir`	no	Working directory for the command

Tooling

`lib/self-healing/cron_manifest.py`

```
# Show what's declared in crons.yaml
python3 cron_manifest.py show crons.yaml

# Diff declared vs live crons
python3 cron_manifest.py diff crons.yaml

# Sync: apply crons.yaml to live system
python3 cron_manifest.py sync crons.yaml

# Run all smoke tests for declared jobs
python3 cron_manifest.py smoke crons.yaml
```

Benefits

1. **Auditable:** Git blame shows who added/changed each job and when
2. **Reproducible:** New environments get the same cron setup
3. **Testable:** Each job has an optional smoke_test command
4. **Documentable:** Description field explains intent
5. **Diffable:** `cron_manifest.py diff` catches drift between declared and actual

Integration with Self-Healing

The smoke test framework (`smoke_runner.sh`) can read `crons.yaml` and run each job's `smoke_test` command as part of the daily health check. If a smoke test fails, the auto-correction loop kicks in.

Spec-Driven Development (SDD)

Formalize the pipeline from idea → specification → implementation plan → task breakdown → execution. Prevents "vibe coding" by making the spec the source of truth, not the code.

Inspired by [GitHub's spec-kit](), adapted for multi-agent orchestration.

Core Idea

Traditional development: idea → code → docs (if lucky). SDD: idea → **constitution** → **spec** → **plan** → **tasks** → code.

Each phase produces a concrete artifact that gates the next phase. Specs are versioned alongside code and evolve with the project.

The Pipeline

```
┌──────────────┐     ┌──────────┐     ┌──────────┐     ┌──────────┐     ┌──────────┐
│ Constitution │──▶│   Spec   │──▶│   Plan   │──▶│  Tasks   │──▶│ Implement │
│  (project)   │     │(feature) │     │(technical)│    │(actionable)│    │  (code)   │
└──────────────┘     └──────────┘     └──────────┘     └──────────┘     └──────────┘
    ▲                                                                          │
    └──────────────── feedback loop (production learnings) ──────────────────┘
```

Phase 1: Constitution

A project-level governance document. Written once, evolves rarely. Every feature must comply.

What it defines:

- Core principles (3-7, not 30)
- Non-negotiable standards (testing, security, observability)
- Technology constraints and preferences
- Quality gates and review process
- Governance rules (how to amend the constitution itself)

Template:

```
# [Project] Constitution

## Core Principles

### I. [Principle Name]
[Description + specific rules]

### II. [Principle Name]
[Description + specific rules]

## Quality Gates
- [ ] Tests pass before merge
- [ ] No [NEEDS CLARIFICATION] markers in specs
- [ ] Constitution compliance verified

## Governance
- Constitution supersedes ad-hoc decisions
- Amendments require documentation + rationale
- Version: X.Y | Ratified: [DATE]
```

Why this matters for agents: Agents without constraints produce inconsistent work. A constitution acts as a persistent system prompt for every agent in the project — it's the "rules" layer from skills-vs-rules.md.

Phase 2: Feature Spec

Describes **what** and **why**, never **how**. Written for business stakeholders, not developers. No tech stack, no APIs, no code structure.

Key innovations from spec-kit worth adopting:

Prioritized User Stories with Independent Testability

Each user story is:

- **Prioritized** (P1, P2, P3) — P1 is your MVP
- **Independently testable** — implementing just P1 gives you a viable product
- **Bounded** with Given/When/Then acceptance scenarios

```
### User Story 1 - Photo Album Creation (Priority: P1) □ MVP

Users can create and name photo albums grouped by date.

**Why this priority**: Core value prop — without albums, there's no product.

**Independent Test**: Upload 3 photos → verify album created with date grouping.

**Acceptance Scenarios**:
1. **Given** a user with photos, **When** they click "Create Album",
   **Then** photos are grouped by date automatically.
2. **Given** an empty album, **When** user drags photos in,
   **Then** album updates with new photos.
```

Requirement Precision

- Use RFC 2119 language (MUST, SHOULD, MAY)
- Max 3 `[NEEDS CLARIFICATION]` markers — force informed guesses over analysis paralysis
- Every requirement must be testable; if you can't test it, rewrite it

Success Criteria (Technology-Agnostic)

Good: "Users complete checkout in under 3 minutes" Bad: "API response time under 200ms" (implementation detail leaking into spec)

Phase 3: Implementation Plan

Translates the spec into technical decisions. This is where tech stack, architecture, and file structure live.

Template structure:

```
# Implementation Plan: [Feature]

## Summary
[What + technical approach]

## Technical Context
- Language/Version: [e.g., TypeScript 5.x]
- Dependencies: [e.g., Next.js, Prisma]
```

```
- Storage: [e.g., PostgreSQL]
- Testing: [e.g., Vitest]

## Constitution Compliance Check
- [ ] Principle I satisfied because [reason]
- [ ] Principle II satisfied because [reason]

## Project Structure
[Concrete file tree — no "Option 1/2/3", pick one]

## Complexity Tracking
| Violation | Why Needed | Simpler Alternative Rejected Because |
|-----------|------------|-------------------------------------|
| [item]    | [reason]   | [why simpler doesn't work]          |
```

Constitution compliance is a gate — plan can't proceed without passing.

Phase 4: Task Breakdown

The most agent-friendly phase. Tasks become directly assignable work units.

Parallelism Markers

```
- [ ] T001 [P] [US1] Create User model in src/models/user.ts
- [ ] T002 [P] [US1] Create Album model in src/models/album.ts
- [ ] T003 [US1] Implement AlbumService in src/services/album.ts (depends on T001, T002)
```

- `[P]` = can run in parallel (different files, no deps)
- `[US1]` = belongs to User Story 1 — enables story-level progress tracking
- **Exact file paths** in every task — no ambiguity for agents

Phased Execution with Checkpoints

```
Phase 1: Setup (project scaffolding)
Phase 2: Foundation (shared infra — BLOCKS all user stories)
   □ Checkpoint: foundation ready
Phase 3: User Story 1 (P1 — MVP)
   □ Checkpoint: US1 independently testable
Phase 4: User Story 2 (P2)
   □ Checkpoint: US1 + US2 both work
Phase N: Polish (cross-cutting concerns)
```

Mapping to Multi-Agent Orchestration

This is where our toolkit adds value over spec-kit:

```yaml
# ChatDev-style DAG for task execution
graph:
  start: [setup]
  nodes:
    - id: setup
      type: agent  # scaffolding agent
    - id: foundation
      type: agent  # core infra agent
    - id: us1_models
```

```
      type: subgraph
      config:
        # Fan-out: parallel model creation
        type: map
        tasks: [T001, T002]  # [P] tasks
    - id: us1_services
      type: agent  # sequential: depends on models
    - id: us1_verify
      type: agent  # checkpoint verification
  edges:
    - from: setup → foundation
    - from: foundation → us1_models
    - from: us1_models → us1_services
    - from: us1_services → us1_verify
```

Tasks marked `[P]` become fan-out nodes. Sequential tasks become edges. User stories become subgraphs. Checkpoints become gate nodes that can pause for human review.

Phase 5: Implementation

Execute tasks phase by phase. Key rules:

- **Mark tasks complete** as you go (`[x]`)
- **Halt on failure** for sequential tasks; continue parallel tasks
- **Validate at checkpoints** before proceeding
- **Checklist gates** can block implementation start (e.g., UX review, security review)

Checklist Gates

Optional but powerful: checklists that must pass before a phase can start.

```
# Pre-Implementation Checklist: Photo Albums

## Spec Quality
- [x] No implementation details in spec
- [x] All acceptance scenarios defined
- [x] Success criteria are measurable + tech-agnostic
- [ ] UX review completed  ← BLOCKS implementation

## Security
- [x] Auth requirements defined
- [x] Data retention policy documented
```

If any checklist item is incomplete, the agent asks before proceeding. This is the "human-in-the-loop checkpoint" pattern applied to specs.

Integration with Existing Patterns

Existing Pattern	How SDD Connects
Phased Execution	SDD formalizes the phases: constitution → spec → plan → tasks → implement
Parallel Agent Teams	`[P]` markers identify parallelizable work; `[US]` markers define team boundaries
Pre-PR Quality Stack	Constitution compliance check = the first quality gate

Safe Looping	Checkpoints between phases = natural bounded iteration points
Skills vs Rules	Constitution = rules (always-on); spec/plan/tasks = skills (per-feature)
ChatDev DAG	Task breakdown maps directly to DAG nodes, edges, and fan-out

When to Use SDD

Good fit:

- Greenfield features with unclear scope
- Multi-agent implementation (tasks need to be unambiguous)
- Projects with quality/compliance requirements
- Anything you'd regret vibe-coding

Overkill:

- Bug fixes, typo corrections
- Well-understood changes to existing patterns
- Exploratory prototypes (spec after, not before)

Directory Convention

```
project/
├── constitution.md          # Project-level (one per project)
└── specs/
    ├── 001-user-auth/
    │   ├── spec.md          # Feature spec
    │   ├── plan.md          # Implementation plan
    │   ├── tasks.md         # Task breakdown
    │   ├── research.md      # Optional: research notes
    │   └── checklists/      # Optional: gate checklists
    │       ├── requirements.md
    │       └── security.md
    └── 002-photo-albums/
        ├── spec.md
        ├── plan.md
        └── tasks.md
```

Branch per feature: `001-user-auth`, `002-photo-albums`. Specs live in the repo, versioned with git.

Key Takeaways

1. **Specs are executable** — they directly generate task DAGs for agent orchestration
2. **Constitution prevents drift** — project principles apply to every feature, every agent
3. **Independent user stories** — each priority level is a standalone MVP increment
4. **Parallelism is explicit** — `[P]` markers + file paths = zero ambiguity for agents
5. **Checkpoints are gates** — phases don't bleed into each other
6. **Feedback loops close** — production learnings update specs, not just code

---

Sources: GitHub spec-kit, spec-driven.md philosophy. Adapted for multi-agent orchestration contexts.

Pre-PR Quality Stack

Run these skills in order before creating any PR. Catches the "works but sloppy" pattern common in AI-generated code.

The Stack

```
1. /code-review  → Self-review for bugs, edge cases, security
2. /simplify     → Can this be done with less code?
3. /deslop       → Remove unnecessary comments, dead code, AI-smell
4. /commit-pr    → Then create the PR
```

Why This Order

1. **Code review first** — Find actual bugs before worrying about style
2. **Simplify second** — Fewer lines = fewer bugs, easier review
3. **Deslop last** — Clean up cosmetic issues after logic is solid
4. **Commit only after all pass** — PR is reviewer-ready

Implementation

As a Compound Skill

Create `skills/pre-pr.md`:

```md
# Pre-PR Quality Check

Before creating any PR, run these steps in order:

1. Review the changes for bugs, edge cases, and security issues
2. Look for opportunities to simplify — can anything be done with less code?
3. Remove AI-smell: unnecessary comments, dead code, overly verbose variable names
4. Only after all checks pass, create the commit and PR
```

As Sequential Skill Invocation

```
"Run /code-review, then /simplify, then /deslop, then /commit-pr"
```

Or configure as a hook that runs before PR creation.

AI-Smell Indicators (for /deslop)

- Comments that just restate the code
- Unused imports or variables
- Overly defensive null checks that can't happen
- Generic variable names (data, result, temp, item)
- Console.log / print statements left in
- TODOs that were already done
- Commented-out code blocks

VSDD-Style Phased Gates

Source: [Verified Spec-Driven Development]

For higher-stakes work, extend the pre-PR stack with formal gates inspired by VSDD:

The Full Pipeline

```
Phase 1: Spec Crystallization
    Behavioral contract + edge case catalog + verification strategy
    Gate: Adversary can't find legitimate holes
        ↓
Phase 2: Test-First Implementation (TDD Core)
    Write ALL tests first → Red Gate (all must fail) → Minimal implementation → Refactor
    Gate: All tests green, human approves alignment with spec intent
        ↓
Phase 3: Adversarial Refinement
    Fresh-context reviewer examines spec + tests + implementation
    Zero-tolerance: every critique is a concrete flaw with location
    Gate: Adversary forced to hallucinate flaws (convergence)
        ↓
Phase 4: Feedback Integration Loop
    Spec flaws → back to Phase 1
    Test flaws → back to Phase 2a
    Implementation flaws → back to Phase 2c
        ↓
Phase 5: Create PR (only after convergence)
```

Key VSDD Ideas Worth Stealing

1. The Adversary Must Have Fresh Context Don't ask the same agent that wrote the code to review it. Use a different session or agent with no accumulated goodwill. Context reset on every review pass.

2. Convergence Signal = Hallucination Point You're done iterating when the reviewer is *forced to invent problems that don't exist*. This is a concrete, testable exit condition — much better than "looks good to me."

3. Spec Review Before Tests The spec is iterated until the adversary can't find legitimate holes in either the behavioral contract or the verification strategy. Tests are never written against a bad spec.

4. Full Traceability

```
Spec Requirement → Test Case → Implementation → Adversarial Review
```

At any point, answer: "Why does this line of code exist?" and trace it to a spec requirement.

When to Use Full VSDD vs Basic Stack

Scenario	Use
Bug fix, small feature	Basic stack (review → simplify → deslop)
New module, API surface	Add spec gate + adversarial review
Security-critical, financial	Full VSDD with formal verification
Prototype/exploration	Skip entirely

Source

Pattern from @leerob (Vercel), adopted by Claude Code team.

Parallel Worktree Operations

Operational pattern for running multiple coding agents on the same codebase without conflicts. Uses git worktrees for isolation, tasklists for coordination, and Graphite for cleanup.

The Problem

Multiple agents working on the same repo will:

- Conflict on file edits (same branch, same files)
- Overwrite each other's uncommitted changes
- Create merge nightmares if not isolated

The Solution: Worktree-Per-Agent

Each parallel coding agent gets its own **git worktree** — a separate checkout of the repo with its own working directory, branch, and index. Changes are isolated until merge.

Convention: Worktrees live in `.trees/` inside the repo (not `/tmp/`). This keeps them discoverable, project-scoped, and survives reboots.

```
main repo: ~/project/                    (orchestrator reads, doesn't write)
    │
    ├── .trees/auth/                      (Agent A's isolated checkout)
    │   └── branch: feat/auth
    ├── .trees/api/                       (Agent B's isolated checkout)
    │   └── branch: feat/api
    └── .trees/ui/                        (Agent C's isolated checkout)
        └── branch: feat/ui
```

See skills/worktrees.md for the full creation skill. See skills/gt-sync-cleanup.md for the cleanup skill.

Lifecycle

1. Create (Before Spawning Agent)

The orchestrator creates worktrees for each agent BEFORE spawning:

```bash
cd ~/project
mkdir -p .trees

# Add .trees to .gitignore
grep -q '^\.trees$' .gitignore 2>/dev/null || echo '.trees' >> .gitignore

# Create worktree per agent
git worktree add -b feat/auth .trees/auth
git worktree add -b feat/api .trees/api
git worktree add -b feat/ui .trees/ui

# Symlink env files + deps for each
for slug in auth api ui; do
  cd .trees/$slug
  for f in ../../.env*; do [ -f "$f" ] && ln -sf "$f" .; done
  [ -d "../../node_modules" ] && ln -sf ../../node_modules .
  cd ../..
done
```

Then spawn each agent with `workdir` set to its `.trees/<slug>` path.

2. Work (Agent Operates in Worktree)

Each agent works in its isolated worktree. It can:

- Read/write files freely
- Run tests, build, lint
- Make commits on its branch
- Push its branch to remote

Agents do NOT interact with each other's worktrees.

3. Submit (Agent Creates PR)

When done, the agent creates a PR from its branch:

```
# Using Graphite
cd /tmp/wt-feature-auth
gt submit

# Or using gh
gh pr create --base main --head feature/auth --title "Implement auth"
```

4. Review & Merge

Orchestrator or reviewer agent:

- Reviews each PR (adversarial review if warranted)
- Merges via GitHub (squash merge preferred for clean history)

5. Cleanup (Post-Merge)

Use the gt-sync-cleanup skill (skills/gt-sync-cleanup.md) for full automated cleanup. The key steps:

1. **Pre-sync:** Detect worktrees with merged/closed PRs via `gh pr view` → remove them (unblocks `gt sync`)
2. **Sync:** `gt sync` fetches remote, restacks branches, deletes merged branch refs
3. **Post-sync:** Remove orphaned worktrees whose branches were deleted by `gt sync`
4. **Prune:** `git worktree prune`

```
# Quick version (from main repo)
cd ~/project

# Pre-sync: remove worktrees for merged PRs
main_wt=$(git rev-parse --show-toplevel)
git worktree list --porcelain | grep '^worktree ' | sed 's/^worktree //' | while read wt; do
  [ "$wt" = "$main_wt" ] && continue
  branch=$(git worktree list | grep "^$wt " | grep -oE '\[.+\]' | tr -d '[]')
  [ -z "$branch" ] && continue
  pr_state=$(gh pr view "$branch" --json state --jq '.state' 2>/dev/null)
  if [ "$pr_state" = "MERGED" ] || [ "$pr_state" = "CLOSED" ]; then
    git worktree remove "$wt" 2>/dev/null && git branch -D "$branch" 2>/dev/null
  fi
done

# Sync
gt sync

# Post-sync: remove orphaned worktrees
git worktree list --porcelain | grep '^worktree ' | sed 's/^worktree //' | while read wt; do
```

```
  [ "$wt" = "$main_wt" ] && continue
  branch=$(git worktree list | grep "^$wt " | grep -oE '\[.+\]' | tr -d '[]')
  [ -z "$branch" ] && continue
  git show-ref --verify --quiet "refs/heads/$branch" || git worktree remove "$wt" 2>/dev/null
done

git worktree prune
```

Safety guarantees:

- Uncommitted changes are protected (`git worktree remove` refuses without `--force`)
- Only MERGED/CLOSED PRs trigger pre-sync removal; OPEN PRs are untouched
- Local-only branches (no PR) are skipped
- Stale git lock files (>5min, no owner process) are cleared before sync

Tasklist Coordination

Use Claude Code's Task tool (or a shared tasklist file) to prevent agents from working on overlapping areas.

Task Assignment File

The orchestrator writes a `TASKS.md` (or uses Claude's built-in Task tool) before spawning agents:

```
# Tasks — Feature Sprint

## Agent A: feature/auth (worktree: /tmp/wt-feature-auth)
- [ ] Implement login flow in src/auth/
- [ ] Add JWT middleware in src/middleware/
- [ ] Write tests in tests/auth/
DO NOT TOUCH: src/api/, src/ui/, src/db/

## Agent B: feature/api (worktree: /tmp/wt-feature-api)
- [ ] Implement REST endpoints in src/api/
- [ ] Add validation schemas in src/api/schemas/
- [ ] Write tests in tests/api/
DO NOT TOUCH: src/auth/, src/ui/, src/middleware/

## Agent C: feature/ui (worktree: /tmp/wt-feature-ui)
- [ ] Build React components in src/ui/
- [ ] Add Storybook stories in src/ui/__stories__/
- [ ] Write tests in tests/ui/
DO NOT TOUCH: src/auth/, src/api/, src/middleware/
```

Key Rules

- **Explicit file boundaries** — each agent knows exactly which directories are theirs
- **DO NOT TOUCH lists** — prevents accidental overlap
- **Shared files** (e.g., types, constants) — either pre-define them before spawning, or assign ONE agent as owner

Handling Shared Dependencies

If agents need shared types/interfaces:

1. **Pre-create shared types** before spawning agents (orchestrator does this)
2. **Or assign one agent as "foundation"** that runs first and creates shared types
3. **Never let two agents edit the same file** — this is the #1 source of merge conflicts

Integration with Tiered Architecture

```
Orchestrator (Pip)
    │
    │ 1. Create worktrees
    │ 2. Write TASKS.md with boundaries
    │ 3. Spawn Claude sub-agents with worktree paths
    │
    ├── Claude: Implementer-A (workdir: .trees/auth)
    │   ├── reads TASKS.md, respects boundaries
    │   └── spawns Codex in same worktree if needed
    │
    ├── Claude: Implementer-B (workdir: .trees/api)
    │   ├── reads TASKS.md, respects boundaries
    │   └── spawns Codex in same worktree if needed
    │
    └── Claude: Reviewer
        └── reviews all PRs post-submission

    After merge:
    │ 4. Run gt-sync-cleanup skill (pre-sync → gt sync → post-sync → prune)
    │ 5. Verify main has all changes
```

Graphite Integration

If using Graphite (`gt`):

```
# Create branches via gt from the worktree
cd /tmp/wt-feature-auth
gt create -am "Implement auth flow"

# Submit PR
gt submit

# After merge, from main repo:
gt sync    # pulls trunk, restacks, cleans merged branches

# Then remove worktrees
git worktree remove /tmp/wt-feature-auth
```

`gt sync` handles branch cleanup automatically for merged branches, but worktree directories still need manual removal.

Anti-Patterns

× **Two agents in the same worktree** — guaranteed conflicts × **No file boundaries** — agents edit the same files, merge hell × **Forgetting cleanup** — orphaned worktrees pile up in /tmp × **Branching off another agent's branch** — use main as base for all worktrees × **Shared mutable state files** — if two agents need to update the same config, serialize (one after the other)

Checklist for Orchestrator

Before spawning parallel coding agents:

- [ ] `mkdir -p .trees` and ensure `.trees` in `.gitignore`
- [ ] Create worktree per agent: `git worktree add -b feat/{slug} .trees/{slug}`
- [ ] Symlink `.env*` files and deps into each worktree
- [ ] Write TASKS.md with explicit file boundaries
- [ ] Pre-create shared types/interfaces if needed
- [ ] Set each agent's workdir to `.trees/{slug}`

After all agents complete and PRs merge:

- [ ] Run gt-sync-cleanup (pre-sync → `gt sync` → post-sync → prune)
- [ ] Verify main branch has all changes
- [ ] `git worktree list` to confirm cleanup

See Also

- Tiered Agent Orchestration — full hierarchy pattern
- Parallel Agent Teams — git-based coordination
- Fleet Parallel Workers — the simpler version

Phased Execution

Break complex tasks into explicit phases. Each phase produces artifacts the next phase consumes.

Source: @pedrohcgs thread, @bcherny tips

Why Phases Beat Monolithic Prompts

Monolithic:

```
Build me a user authentication system with magic links,
session management, and protected routes.
```

→ Agent tries to do everything at once, gets confused, produces incomplete work.

Phased:

```
Phase 1: Research existing auth patterns in this codebase
Phase 2: Write spec for magic link auth
Phase 3: Create implementation plan with file list
Phase 4: Implement auth module (following plan)
Phase 5: Add tests
Phase 6: Integration testing
```

→ Each phase has clear deliverable. Agent can verify completion before moving on.

The Standard Phases

```
┌──────────────┐
│  RESEARCH    │  Gather context, understand constraints
└──────┬───────┘
       ↓
┌──────────────┐
│    SPEC      │  Define what we're building (acceptance criteria)
└──────┬───────┘
```

```
        ↓
┌──────────────┐
│    PLAN      │  Break into tasks, identify files, sequence work
└──────────────┘
        ↓
┌──────────────┐
│  EXECUTE     │  Implement (possibly in sub-phases)
└──────────────┘
        ↓
┌──────────────┐
│   VERIFY     │  Test, review, validate against spec
└──────────────┘
```

Phase Artifacts

Each phase produces a concrete artifact:

Phase	Artifact	Example
Research	Context doc	`docs/auth-research.md`
Spec	Requirements	`docs/auth-spec.md`
Plan	Task breakdown	`docs/auth-plan.md` or GitHub issues
Execute	Code	`src/auth/*`
Verify	Test results	CI green, manual QA notes

Implementation Patterns

Pattern 1: Plan Mode → Execute Mode

Claude Code's built-in approach:

```
1. Start in Plan mode (Shift+Tab twice)
2. Iterate on plan until solid
3. Accept plan → auto-clears context
4. Execute with fresh context focused on the plan
```

Key insight: Accepting a plan clears context, so the execution gets a clean slate with just the plan.

Pattern 2: Explicit Phase Prompts

```
## Phase 1: Research
Read the existing codebase and document:
- Current auth approach (if any)
- Database schema for users
- Existing session handling
- Related routes

Output: Write findings to docs/auth-research.md

## Phase 2: Spec
Based on research, write a spec for magic link auth:
- User stories
- Acceptance criteria
```

```
- Edge cases
- Security considerations

Output: Write to docs/auth-spec.md

[continue for each phase...]
```

Pattern 3: Phase Gates

Require human approval between phases:

```
After completing each phase:
1. Commit your work
2. Push and stop
3. Wait for approval to continue to next phase

Phase gates:
- Research → Spec: Approval needed
- Spec → Plan: Approval needed
- Plan → Execute: Approval needed
- Execute → Verify: Auto-continue
```

Pattern 4: Subagent Per Phase

Different agents for different phases:

```
// Research agent (read-only, web access)
await sessions_spawn({
  task: "Research auth patterns. Output to docs/auth-research.md",
  label: "auth-research"
});

// Planning agent (architect)
await sessions_spawn({
  task: "Read docs/auth-research.md. Create implementation plan.",
  label: "auth-plan"
});

// Implementation agent (coder)
await sessions_spawn({
  task: "Read docs/auth-plan.md. Implement following the plan exactly.",
  label: "auth-implement"
});
```

Spec Document Template

```
# Feature: [Name]

## Overview
[One paragraph summary]

## User Stories
- As a [user], I want [capability] so that [benefit]
- ...
```

```
## Acceptance Criteria
- [ ] [Specific testable criterion]
- [ ] [Specific testable criterion]
- [ ] ...

## Technical Approach
[High-level approach]

## Files to Create/Modify
- `src/auth/magic-link.ts` - Core logic
- `src/auth/session.ts` - Session management
- `src/routes/auth.ts` - API routes
- `tests/auth.test.ts` - Tests

## Edge Cases
- [ ] Expired magic links
- [ ] Multiple devices
- [ ] Rate limiting

## Security Considerations
- Token expiry
- Single-use links
- IP validation (optional)

## Out of Scope
- OAuth providers (future)
- 2FA (future)
```

Plan Document Template

```
# Implementation Plan: [Feature]

## Phase 1: [Name]
**Goal:** [What this phase achieves]
**Files:**
- [ ] `path/to/file.ts` - [what to do]
- [ ] `path/to/file.ts` - [what to do]
**Tests:** [How to verify phase is complete]
**Estimated:** [X commits]

## Phase 2: [Name]
...

## Dependencies
- Phase 2 depends on Phase 1
- Phase 3 can run in parallel with Phase 2

## Risks
- [Risk and mitigation]
```

Anti-Patterns

× **Skipping research** — agent makes assumptions × **Vague specs** — "make it work" isn't a spec × **No plan** — agent wanders × **Executing before plan approval** — expensive mistakes × **Phases too granular** — overhead exceeds benefit

When to Use

□ Features (multi-file changes) □ Refactors (high risk of regression) □ Integrations (external dependencies) □ Anything >2 hours of work

× **Bug fixes (usually single phase)** × **Typos/docs (just do it)** × **Exploratory work (research IS the work)**

See Also

- patterns/safe-looping.md — bounded execution within phases
- patterns/parallel-agent-teams.md — multiple agents per phase

---

Hooks as Middleware

Hooks let you intercept agent actions before or after execution — like middleware for your AI agent.

Source: @adocomplete

Core Concept

```
User Request
    ↓
┌──────────────────┐
│  PreToolUse      │  ← Intercept BEFORE tool runs
│  (validate,      │
│   transform,     │
│   log)           │
└──────────────────┘
         │
         ↓
┌──────────────────┐
│  Tool Executes   │
└──────────────────┘
         │
         ↓
┌──────────────────┐
│  PostToolUse     │  ← Intercept AFTER tool runs
│  (verify,        │
│   transform,     │
│   trigger next)  │
└──────────────────┘
         │
         ↓
    Agent Response
```

Use Cases

1. Logging & Observability

```yaml
hooks:
  PostToolUse:
```

```
    - command: "echo '[$(date)] Tool: $TOOL_NAME' >> ~/.agent_logs/tool_calls.log"
      async: true  # Don't block agent
```

2. Validation / Guardrails

```
hooks:
  PreToolUse:
    - matcher: "Bash"
      command: |
        # Block dangerous commands
        if echo "$TOOL_INPUT" | grep -qE "rm -rf|DROP TABLE|curl.*\| bash"; then
          echo "BLOCKED: Dangerous command detected"
          exit 1
        fi
```

3. Auto-Format After Edits

```
hooks:
  PostToolUse:
    - matcher: "Edit|Write"
      command: "pnpm prettier --write $FILE_PATH"
      async: true
```

4. Test After Code Changes

```
hooks:
  PostToolUse:
    - matcher: "Edit|Write"
      filter: "**/*.ts"
      command: "pnpm test --related $FILE_PATH"
```

5. Notifications

```
hooks:
  PostToolUse:
    - matcher: "Task"  # When a task/subagent completes
      command: "notify-send 'Agent completed task: $TASK_DESCRIPTION'"
      async: true
```

6. Auto-Commit Checkpoints

```
hooks:
  PostToolUse:
    - matcher: "Edit|Write"
      command: |
        cd $PROJECT_ROOT
        git add -A
        git commit -m "checkpoint: $TOOL_NAME on $FILE_PATH" --allow-empty
      async: true
```

7. Context Injection

```yaml
hooks:
  PreToolUse:
    - matcher: "Read"
      command: |
        # Add related files to context
        if [[ "$FILE_PATH" == *.test.* ]]; then
          echo "Also consider: ${FILE_PATH/.test/}"
        fi
```

Claude Code Configuration

In `~/.claude/settings.json` or project `.claude/settings.json`:

```json
{
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "command": "./scripts/validate-command.sh"
      }
    ],
    "PostToolUse": [
      {
        "matcher": "Edit|Write",
        "command": "pnpm lint:fix $FILE_PATH",
        "async": true
      }
    ],
    "Stop": [
      {
        "command": "./scripts/on-agent-stop.sh"
      }
    ]
  }
}
```

Available Hooks

- `PreToolUse` — before any tool call
- `PostToolUse` — after any tool call
- `Stop` — when agent stops (manually or naturally)
- `Notification` — when agent wants to notify user
- `TeammateIdle` — when a teammate agent becomes idle (agent teams)
- `TaskCompleted` — when a spawned task finishes

Environment Variables Available

- `$TOOL_NAME` — which tool is running
- `$TOOL_INPUT` — the input to the tool
- `$FILE_PATH` — for file operations
- `$PROJECT_ROOT` — project root directory
- `$SESSION_ID` — current session ID

Async Hooks

Add `async: true` for hooks that shouldn't block:

- Logging
- Notifications
- Non-critical formatting

Keep synchronous for:

- Validation/guardrails
- Required verification

Patterns

The Guardrail Stack

```
PreToolUse:
  - matcher: "Bash"
    command: "./hooks/validate-bash.sh"      # Block dangerous commands
  - matcher: "Write"
    command: "./hooks/validate-write.sh"     # Block writes to protected paths
  - matcher: "Edit"
    command: "./hooks/validate-edit.sh"      # Require backup before edit
```

The Quality Stack

```
PostToolUse:
  - matcher: "Edit|Write"
    filter: "**/*.{ts,tsx,js,jsx}"
    command: "pnpm eslint --fix $FILE_PATH"
    async: true
  - matcher: "Edit|Write"
    filter: "**/*.{ts,tsx,js,jsx}"
    command: "pnpm prettier --write $FILE_PATH"
    async: true
```

The Audit Trail

```
PreToolUse:
  - command: |
      echo "{\"ts\": \"$(date -Iseconds)\", \"tool\": \"$TOOL_NAME\", \"phase\": \"pre\"}" >>
~/.agent_audit.jsonl
    async: true
PostToolUse:
  - command: |
      echo "{\"ts\": \"$(date -Iseconds)\", \"tool\": \"$TOOL_NAME\", \"phase\": \"post\"}" >>
~/.agent_audit.jsonl
    async: true
```

Anti-Patterns

× **Slow synchronous hooks** — blocks agent on every tool call × **Hooks that fail silently** — agent continues unaware × **Too many hooks** — death by a thousand cuts to latency × **Hooks with side effects on PreToolUse** —

tool might not execute

Testing Hooks

```
# Test a hook script manually
TOOL_NAME="Bash" TOOL_INPUT="rm -rf /" ./hooks/validate-bash.sh
echo $?  # Should be non-zero (blocked)

TOOL_NAME="Bash" TOOL_INPUT="ls -la" ./hooks/validate-bash.sh
echo $?  # Should be 0 (allowed)
```

See Also

- Claude Code Hooks Documentation
- patterns/pre-pr-quality-stack.md — example quality workflow

---

CLI Over MCP

Source: Eric Holmes — "MCP is dead. Long live the CLI"

Core Argument

When a CLI exists for a service, prefer it over MCP. LLMs don't need a special protocol to use tools — they've trained on millions of man pages, Stack Overflow answers, and shell scripts.

Why CLIs Win

1. Composability

CLIs compose. Pipe through `jq`, chain with `grep`, redirect to files.

```
# CLI: powerful, composable
terraform show -json plan.out | jq '[.resource_changes[] | select(.change.actions[0] == "no-op"
| not)] | length'

# MCP: dump entire plan into context (expensive, often impossible)
# or build custom filtering into the MCP server
```

2. Debuggability

When Claude does something unexpected with `gh pr view 123`, you can run the same command and see exactly what it saw. Same input, same output, no mystery.

With MCP, the tool only exists inside the LLM conversation. Debugging requires spelunking through JSON transport logs.

3. Auth Already Works

`aws` uses profiles and SSO. `gh` uses `gh auth login`. `kubectl` uses kubeconfig. Battle-tested auth flows that work the same whether a human or an agent is driving.

MCP reinvents auth per-server. When it breaks, you need MCP-specific troubleshooting.

4. No Moving Parts

CLI tools are binaries on disk. No background processes, no state management, no initialization dance.

MCP servers are processes that need to start up, stay running, and not silently hang. Initialization is flaky. Re-auth never ends.

5. Permission Granularity

With CLIs, you can allowlist `gh pr view` but require approval for `gh pr merge`. MCP permissions are typically all-or-nothing per tool name.

When MCP Makes Sense

- Tool genuinely has no CLI equivalent
- The service is inherently stateful/streaming (e.g., database connections)
- You need real-time push notifications from a service
- The standardized interface matters more than composability

Decision Framework

```
Does a CLI exist for this service?
    ├── Yes → Use the CLI
    │         (even if an MCP server also exists)
    └── No
        ├── Can you use a REST API via curl? → Use curl
        └── Neither exists → MCP is appropriate
```

For Tool Builders

If you're building an MCP server but don't have an official CLI: stop and rethink. Ship a good API, then ship a good CLI. The agents will figure it out.

See Also

- harness-engineering.md — "See Like an Agent" tool design principles
- skills-vs-rules.md — when to add tools vs instructions

Runbook: Spin Up a Discord-Driven Autonomous Agent CEO

Step-by-step guide to create a new autonomous AI agent with its own OpenClaw instance, Discord command center, nightly self-improvement, and revenue tracking. Based on our Roam setup (March 2026).

Prerequisites

- OpenClaw installed and running (`openclaw status`)
- A Discord account to create the bot
- A GitHub account for the workspace repo
- ~15 minutes

Step 1: Create Discord Bot

1. Go to https://discord.com/developers/applications
2. Click **New Application** → name it (e.g. "Roam")

3. Go to **Bot** tab → click **Reset Token** → copy the token
4. Under **Privileged Gateway Intents**, enable:
   - ☐ Message Content Intent
   - ☐ Server Members Intent
5. Go to **OAuth2** → URL Generator:
   - Scopes: `bot`, `applications.commands`
   - Bot Permissions: Administrator (or at minimum: View Channels, Send Messages, Read Message History, Embed Links, Attach Files, Add Reactions, Manage Channels)
6. Copy the generated URL → open it → invite bot to your server
7. Note your **Server (Guild) ID**: right-click server icon → Copy Server ID (enable Developer Mode in Discord Settings → Advanced first)

You'll need:

- Bot token: `MTQ3...` (long string)
- Server/Guild ID: `1478778032465510502` (numeric)
- App ID: `1478778334824501339` (numeric, from General Information)

Step 2: Create GitHub Repo

```
# On GitHub: create empty repo (e.g. apfong/roam)
# Locally:
mkdir -p ~/agent-name
cd ~/agent-name
git init && git checkout -b main
git remote add origin https://github.com/YOUR_USER/REPO.git
```

Step 3: Create Agent Workspace

The workspace needs these files at minimum:

SOUL.md (identity + operating principles)

```
# SOUL.md — AgentName

You are AgentName — an autonomous AI agent running a business.
You are not an assistant waiting for instructions. You are a founder with a job.

## Core Principles
- Revenue > vanity. Every action traces to making money or reducing costs.
- Ship > perfect. Get things in front of customers fast.
- Automate yourself. If you do something twice, build a system.
- Pay for yourself. Revenue must exceed costs.

## Boundaries
- Never send money without explicit approval
- Be upfront about being AI in customer interactions
- When uncertain about public comms, draft and wait for approval

## Communication
- Discord = primary command center
```

```
- Post what you're doing to relevant channels
- React-based approvals: □ = go, x = revise
```

IDENTITY.md

```
# IDENTITY.md
- **Name:** AgentName
- **Role:** Autonomous AI Founder / CEO
- **Emoji:** □
```

USER.md

```
# USER.md
- **Name:** YourName
- **Role:** Founder/Investor
- **Timezone:** America/New_York
- ***Availability:*** Checks Discord 1-2x daily
- ***Escalate when:*** Spending >$50, first-time public comms, irreversible decisions
```

AGENTS.md (operating manual)

```
# AGENTS.md

## Every Session — Orient → Work → Persist
### Orient
1. Read SOUL.md, USER.md, goals.md
2. Read memory/daily/YYYY-MM-DD.md (today + yesterday)
3. Check Discord channels for pending items

### Work
- Default: find highest-value work available
- Log all significant actions to daily notes

### Persist
- Update goals.md, write daily notes, git commit + push
```

goals.md

```
# Goals — Active Threads
## □ Active
- [ ] First product decision
- [ ] Landing page
- [ ] Stripe integration
```

TOOLS.md (operational notes — fill in after setup)

```
# TOOLS.md
## Discord Command Center
- Server ID: GUILD_ID
- Channel IDs: (filled in Step 5)
```

```
## Stripe
- Secret key: ~/.config/stripe/secret_key
- Publishable key: ~/.config/stripe/publishable_key
```

Directory structure

```
mkdir -p memory/{knowledge-graph,daily,tacit,notes,observations}
mkdir -p scripts products finances
```

finances/costs.md

```
# Cost Tracking
| Item | Cost | Notes |
|------|------|-------|
| Claude API | ~$TBD/mo | |
| Hosting | $0 | Running on host machine |

## Revenue
| Date | Source | Amount |
|------|--------|--------|

## Net: $0
```

scripts/nightly-improve.md

```
# Nightly Self-Improvement Protocol
1. Check if improvement log exists for today (skip if backup run)
2. Review today's session transcripts
3. Extract: regressions → AGENTS.md, patterns → memory/tacit/
4. Write changelog to memory/daily/YYYY-MM-DD-improvements.md
5. Git commit + push
6. Post summary to Discord #daily-report
```

Commit and push

```
cd ~/agent-name
git add -A
git commit -m "Initial workspace: identity, memory, goals, finances"
git push -u origin main
```

Step 4: Create OpenClaw Agent Instance

```
openclaw agents add AGENT_NAME \
  --workspace ~/agent-name \
  --model anthropic/claude-opus-4-6 \
  --non-interactive
```

This creates:

- Agent config in `~/.openclaw/agents/AGENT_NAME/`
- Session storage separate from main agent
- Own auth profile (inherits from main if not overridden)

Verify: `openclaw agents list`

Step 5: Configure Discord Channel + Binding

Add Discord to OpenClaw config

Use `gateway config.patch` (from agent tool or CLI):

```
{
  "channels": {
    "discord": {
      "enabled": true,
      "token": "YOUR_BOT_TOKEN",
      "groupPolicy": "open",
      "replyToMode": "first",
      "historyLimit": 20,
      "dmPolicy": "pairing",
      "guilds": {
        "YOUR_GUILD_ID": {
          "requireMention": false
        }
      },
      "activity": "Building autonomously",
      "activityType": 4,
      "status": "online"
    }
  },
  "plugins": {
    "entries": {
      "discord": {
        "enabled": true
      }
    }
  },
  "bindings": [
    {
      "agentId": "AGENT_NAME",
      "match": {
        "channel": "discord"
      }
    }
  ]
}
```

Critical settings:

- `requireMention: false` — without this, the bot ignores messages unless @mentioned
- `bindings` — routes all Discord traffic to your new agent (not the main/default agent)
- `activityType: 4` — custom status shown in Discord

If you already have Discord configured for another agent, use guild-specific or channel-specific bindings instead of a blanket channel binding:

```
{
  "bindings": [
    {
      "agentId": "agent-a",
      "match": { "channel": "discord", "guildId": "GUILD_A_ID" }
    },
    {
      "agentId": "agent-b",
      "match": { "channel": "discord", "guildId": "GUILD_B_ID" }
    }
  ]
}
```

Restart gateway

The config.patch triggers a restart automatically. Verify:

```
openclaw channels status
# Should show: Discord default: enabled, configured, running, bot:@AgentName
```

Troubleshooting:

- `error: 4014` → Enable Message Content Intent in Developer Portal
- `reason: no-mention` in logs → Set `requireMention: false` in guild config
- Bot shows `stopped` → Check token, check intents, restart gateway

---

Step 6: Create Discord Channels

Use the `message` tool (from the agent) or API:

```
# Create categories
message(action="category-create", channel="discord", guildId="GUILD_ID", name="□ Operations")
message(action="category-create", channel="discord", guildId="GUILD_ID", name="□ Revenue")
message(action="category-create", channel="discord", guildId="GUILD_ID", name="□ Development")
message(action="category-create", channel="discord", guildId="GUILD_ID", name="□ Content")

# Create channels under categories (use category ID from response)
message(action="channel-create", channel="discord", guildId="GUILD_ID", name="daily-report",
topic="Nightly summaries", parentId="CAT_ID")
message(action="channel-create", ..., name="support", topic="Customer issues, escalations")
message(action="channel-create", ..., name="decisions", topic="Needs □/× approval")
message(action="channel-create", ..., name="sales", topic="Outreach, leads")
message(action="channel-create", ..., name="finances", topic="Revenue, costs, P&L")
message(action="channel-create", ..., name="dev-log", topic="Code changes, deployments")
message(action="channel-create", ..., name="bugs", topic="Auto-evaluated bug reports")
message(action="channel-create", ..., name="twitter-drafts", topic="Draft tweets — □ to post, ×
to revise")
message(action="channel-create", ..., name="blog", topic="Draft blog posts")
```

Save all channel IDs to TOOLS.md — the agent needs them for targeted posting.

Step 7: Set Up Nightly Self-Improvement Crons

Two crons per agent — primary + backup (crons fail ~10-15% of the time):

```
# Primary: 2am ET
openclaw cron add \
  --agent AGENT_NAME \
  --name "AgentName Nightly Self-Improvement (Primary)" \
  --cron "0 2 * * *" \
  --tz "America/New_York" \
  --session isolated \
  --announce \
  --channel discord \
  --to "channel:DAILY_REPORT_CHANNEL_ID" \
  --timeout-seconds 1800 \
  --message "Run your nightly self-improvement protocol. Read scripts/nightly-improve.md. Review today's sessions, improve memory/skills/templates, write changelog to memory/daily/YYYY-MM-DD-improvements.md, git commit and push."

# Backup: 3am ET
openclaw cron add \
  --agent AGENT_NAME \
  --name "AgentName Nightly Self-Improvement (Backup)" \
  --cron "0 3 * * *" \
  --tz "America/New_York" \
  --session isolated \
  --announce \
  --channel discord \
  --to "channel:DAILY_REPORT_CHANNEL_ID" \
  --timeout-seconds 1800 \
  --message "BACKUP: Check if today's improvement log exists in memory/daily/. If already done, say 'Already completed.' Otherwise run the full protocol from scripts/nightly-improve.md."
```

Verify: `openclaw cron list` — should show both with `agentId: AGENT_NAME`

Step 8: Configure Stripe (Optional)

```
mkdir -p ~/.config/stripe
echo "sk_test_..." > ~/.config/stripe/secret_key
echo "pk_test_..." > ~/.config/stripe/publishable_key
```

For production, use live keys (`sk_live_...`). Test mode is fine for development.

Step 9: Verify Everything Works

1. **Post in Discord** → agent should respond (check `openclaw channels logs` if not)
2. **Check cron list** → `openclaw cron list` shows agent's crons
3. **Check agent list** → `openclaw agents list` shows the agent
4. **Git repo** → workspace pushed and accessible
5. **Send intro message** — trigger via one-shot cron:

```
openclaw cron add \
  --agent AGENT_NAME \
  --name "First Boot" \
  --every 60m \
  --session isolated \
  --no-deliver \
```

```
  --timeout-seconds 120 \
  --message "Send intro message to Discord #general (channel:GENERAL_ID). Introduce yourself, explain channels, then post initial status to #daily-report."

# Trigger immediately
openclaw cron run JOB_ID

# Remove after it fires
openclaw cron rm JOB_ID
```

Multi-Agent Setup (Multiple Discord-Driven Agents)

Each autonomous agent needs:

- Its own Discord server (simplest) OR separate guild bindings in one server
- Its own OpenClaw agent instance (`openclaw agents add`)
- Its own workspace directory and git repo
- Its own nightly improvement crons

For multiple agents sharing one OpenClaw gateway, route via guild-specific bindings:

```
{
  "bindings": [
    { "agentId": "ceo-agent", "match": { "channel": "discord", "guildId": "111..." } },
    { "agentId": "sales-agent", "match": { "channel": "discord", "guildId": "222..." } },
    { "agentId": "dev-agent", "match": { "channel": "discord", "guildId": "333..." } }
  ]
}
```

Or use a single server with channel-specific routing (advanced — requires per-channel session keys which Discord does automatically).

Gotchas & Lessons Learned

1. **`requireMention` defaults to true** — your bot will silently ignore messages without it. Always set `requireMention: false` in guild config.
2. **Message Content Intent must be enabled** in Discord Developer Portal, not just the config. Error 4014 = this.
3. **Cross-context messaging is blocked** — Pip (Telegram-bound) can't send to Discord. Only the Discord-bound agent (Roam) can. Use cron jobs to trigger the right agent.
4. **`sessions_spawn` can't target other agents** unless explicitly allowed in config. Use cron jobs + `openclaw cron run` for cross-agent triggering.
5. **SSH vs HTTPS for git** — if SSH keys aren't set up, use HTTPS remote URLs.
6. **Backup crons are essential** — primary crons fail ~10-15% of the time. Always have a 3am backup that checks if the 2am run completed.
7. **Bot token in config is redacted** in `config.get` output — you can't read it back. Keep a copy somewhere safe.

Reference: Roam's Setup (First Implementation)

- **Agent:** `roam`
- **Workspace:** `~/roam`
- **Repo:** `github.com/apfong/roam`
- **Discord Guild:** `1478778032465510502`

- **Bot App ID:** `1478778334824501339`
- **Model:** `anthropic/claude-opus-4-6`
- **Crons:** 2am primary + 3am backup (both `agentId: roam`, deliver to Discord)
- **Channels:** general, daily-report, support, decisions, sales, finances, dev-log, bugs, twitter-drafts, blog

---

Harness Engineering

How to systematically improve your AI agent's effectiveness over time.

Source: [Mitchell Hashimoto - My AI Adoption Journey]

Core Principle

> *"Every agent mistake → new test or AGENTS.md fix"*

The agent harness (prompts, rules, tests, tools) is as important as the code. Engineer it like production software.

The Feedback Loop

```
Agent makes mistake
        ↓
┌──────────────────────┐
│  Diagnose root cause  │
│  - Unclear prompt?    │
│  - Missing context?   │
│  - Wrong tool access? │
│  - Missing test?      │
└──────────────────────┘
            │
            ↓
┌──────────────────────┐
│  Fix the harness      │
│  - Update CLAUDE.md   │
│  - Add test case      │
│  - Improve skill      │
│  - Restrict tool      │
└──────────────────────┘
            │
            ↓
Agent less likely to repeat mistake
```

The Six-Step Progression

Mitchell's journey from skeptic to power user:

1. Drop the Chatbot Mindset

Stop treating AI as a Q&A chatbot. Use **agents** that can take action.

Before: Ask Claude a question, copy-paste the answer **After:** Give Claude a task, let it edit files directly

2. Reproduce Your Own Work

Do tasks manually AND with the agent. Compare results. Learn the edges.

```
Task: Add error handling to this function
```

```
Manual: Takes 10 minutes, handles 5 edge cases
Agent: Takes 2 minutes, handles 3 edge cases

→ Learn which edge cases agent misses
→ Add to CLAUDE.md: "Always handle: [those edge cases]"
```

3. End-of-Day Agents

Before signing off, kick off long-running agent tasks:

- Research for tomorrow's work
- Triage issues/PRs
- Run analysis that needs time

Why: Agent works while you don't. Review results next morning.

4. Outsource Slam Dunks

Identify high-confidence tasks agent handles well. Let it do those in background.

```
## Slam Dunks (agent handles autonomously)
- Test file generation
- Documentation updates
- Simple bug fixes with clear repro
- Dependency updates with passing tests
- Code formatting/linting fixes
```

5. Engineer the Harness

Every mistake becomes improvement:

Mistake Type	Harness Fix
Style violation	Add to rules
Missed edge case	Add test
Wrong approach	Document in CLAUDE.md
Used wrong tool	Restrict tool access
Missed context	Improve skill/prompt

CLAUDE.md evolution:

```
# CLAUDE.md

## Lessons Learned (agent wrote these)
- Always check for null before accessing .length
- The payments module uses Stripe, not our internal billing
- Tests must pass before any PR is created
- When editing auth code, also update the auth tests
```

6. Always Have an Agent Running

Target: 10-20% of workday has an agent actively working.

Pro tip: Turn off desktop notifications. You control when to check on the agent, not the other way around.

Implementation Checklist

Weekly Harness Review

- [ ] Review agent mistakes from the week
- [ ] Update CLAUDE.md with new rules
- [ ] Add tests for failure cases
- [ ] Refine skills that performed poorly
- [ ] Document failed approaches (so agent doesn't repeat)

Mistake Triage Template

```
## Mistake: [what happened]
**Date:** YYYY-MM-DD
**Task:** [what agent was trying to do]
**Expected:** [what should have happened]
**Actual:** [what actually happened]
**Root cause:** [unclear prompt / missing context / wrong tool / etc.]
**Fix applied:** [what you changed in the harness]
```

CLAUDE.md Structure for Harness Engineering

```
# Project: [name]

## Context
[What this project is, key technologies]

## Rules (Always)
[Style, conventions, security]

## Learned Rules (From Mistakes)
- [Rule added after mistake 1]
- [Rule added after mistake 2]

## Failed Approaches (Don't Repeat)
- Tried X for Y, failed because Z
- Library A doesn't work with B

## Slam Dunks (Agent Handles Well)
- [Task type 1]
- [Task type 2]

## Needs Human (Agent Struggles)
- [Task type that needs oversight]
- [Task type that needs domain knowledge]
```

Metrics to Track

- **Mistake rate:** How often does agent need correction?
- **Harness growth:** Lines added to CLAUDE.md per week
- **Slam dunk coverage:** % of tasks agent handles autonomously
- **Time to first useful output:** How long until agent produces something usable?

"See Like an Agent" — Tool Design Principle

Source: Thariq (@trq212) — "Lessons from Building Claude Code"

The Claude Code team's core framework for designing agent tools:

Shape Tools to Model Abilities

Imagine being given a difficult math problem. What tools would you want?

- Paper = minimum (limited by manual calculation)
- Calculator = better (but need to know the interface)
- Computer = most powerful (but need to know how to code)

Design tools that match what the model is good at. Not what's easiest to implement. Not what a human would want.

How to Learn What Works

"You pay attention, read its outputs, experiment." There are no rigid rules. It depends on:

- The model you're using (different models need different tools)
- The goal of the agent
- The environment it operates in

Lessons from Claude Code's Tool Evolution

1. AskUserQuestion Tool (Elicitation)

- Attempt 1: Added questions as parameter to ExitPlanTool → confused the model
- Attempt 2: Modified markdown output format for inline questions → unreliable formatting
- Attempt 3: Dedicated tool that triggers a UI modal → worked well, Claude liked calling it
- **Takeaway:** Even the best-designed tool doesn't work if the model doesn't understand how to call it

2. TodoWrite → Task Tool (Capability Drift)

- Original: TodoWrite tool + system reminders every 5 turns
- Problem: As models improved, reminders made Claude think it had to stick to the list rigidly
- Fix: Replaced with Task Tool — supports dependencies, shared updates across subagents, model can alter/delete
- **Takeaway:** Tools that worked for weaker models may constrain stronger ones. Revisit assumptions as capabilities change.

3. RAG → Grep → Progressive Disclosure (Context Building)

- V1: RAG vector database (fragile, needed indexing, context given TO Claude)
- V2: Grep tool (Claude builds its OWN context by searching)
- V3: Skills that reference other files recursively (nested search, progressive discovery)
- **Takeaway:** As models get smarter, let them build their own context instead of pre-computing it for them

Progressive Disclosure Pattern

Add capabilities without adding tools:

- Skill files reference deeper context files
- Model reads skill → discovers references → reads those → discovers more
- Each layer only loads when needed (not upfront in system prompt)

Example: Instead of putting all Claude Code docs in the system prompt, Claude Code gives the model a link to its docs. When asked about itself, it loads the Guide subagent which has instructions on searching docs well.

Rule: Every line in your system prompt/CLAUDE.md should answer: "Would the agent make a mistake if this line wasn't here?" If no, cut it.

Applying to Our Stack

1. When Codex or sub-agents struggle with a tool, ask: is the tool shaped wrong for the model?
2. Periodically audit AGENTS.md/CLAUDE.md — does every line prevent a real mistake?
3. Prefer progressive disclosure (reference files) over front-loading all context
4. When adding a capability, try adding a file/skill reference before adding a tool

---

Anti-Patterns

× **Accepting mistakes without fixing harness** — same mistakes repeat × **Blaming the model** — usually it's the harness × **Fixing mistakes in code only** — should also fix in harness × **Huge CLAUDE.md** — split into skills (see skills-vs-rules.md)

Mental Model

Think of the harness as **institutional knowledge for your agent**.

Humans learn from mistakes and remember. Agents don't — unless you encode the learning in the harness.

Your job shifts from "doing the work" to "teaching the agent to do the work."

See Also

- patterns/safe-looping.md — bounded agent execution
- patterns/skills-vs-rules.md — when to use each
- Mitchell's full post

ChatDev 2.0 YAML Workflow Patterns

Source: https://github.com/OpenBMB/ChatDev (cloned to `chatdev-2.0/`)

Core Schema

```yaml
version: 0.4.0
vars:
  MODEL_NAME: gpt-4o
  API_KEY: ${API_KEY}  # env var substitution

graph:
  id: workflow_name
  description: What this workflow does
  log_level: DEBUG|INFO
  is_majority_voting: false
  initial_instruction: Optional user-facing prompt
  start: [node_ids...]  # entry points (can be multiple for parallel start)
  end: [node_ids...]    # terminal nodes
  nodes: [...]
  edges: [...]
  memory: []  # optional memory configs
```

Node Types

1. Agent Node (LLM call)

```yaml
- id: Planner
  type: agent
  context_window: 20  # how many messages to keep (-1 = all, 0 = none)
  config:
    provider: openai|gemini|anthropic
    name: gpt-4o
    base_url: ${BASE_URL}
    api_key: ${API_KEY}
    role: |
      System prompt goes here...
    params:
      temperature: 0.1
      max_tokens: 4000
    tooling:
      - type: function
        config:
          tools:
            - name: web_search
            - name: deep_research:All  # tool groups
          timeout: null
    thinking: null  # for o1-style reasoning
    memories: []
    retry: null
```

2. Literal Node (static content)

```yaml
- id: UserPrompt
  type: literal
  config:
    content: "Please plan what to eat in Shanghai"
    role: user|assistant
```

3. Passthrough Node (aggregation/routing)

```yaml
- id: Aggregator
  type: passthrough
  config:
    only_last_message: false  # true = only forward last msg
```

4. Subgraph Node (nesting)

```yaml
- id: ReActLoop
  type: subgraph
  config:
    type: file
    config:
      path: "subgraphs/react_agent.yaml"
```

```yaml
# Or inline:
- id: InlineCritic
  type: subgraph
  config:
    type: config
    config:
      id: critic_graph
      nodes: [...]
      edges: [...]
```

5. Loop Counter Node

```yaml
- id: LoopGate
  type: loop_counter
  config:
    max_iterations: 3
    reset_on_emit: true
    message: "Loop finished after three passes"
```

Edge Configuration

```yaml
edges:
  - from: NodeA
    to: NodeB
    trigger: true          # whether this edge activates NodeB
    condition: 'true'      # or condition object below
    carry_data: true       # pass messages along
    keep_message: false    # retain in context
    clear_context: false   # reset node's context
    process: null          # optional transform
    dynamic: null          # for fan-out (see below)
```

Conditional Routing (keyword-based)

```yaml
condition:
  type: keyword
  config:
    any:
      - "ROUTE: Planner"
      - "FINISHED"
    none: []
    regex: []
    case_sensitive: true
```

Dynamic Execution (Fan-Out/Fan-In)

Map Pattern (parallel execution per item)

```yaml
dynamic:
  type: map
  split:
```

```
    type: regex
    config:
      pattern: "<Query>:\\s*(.*)"  # extract each query
  config:
    max_parallel: 3
```

Tree Pattern (hierarchical aggregation)

```
dynamic:
  type: tree
  split:
    type: message
  config:
    group_size: 2      # aggregate in pairs
    max_parallel: 10
```

Pattern Library

1. Deep Research (Iterative Chapter Writing)

Flow: START → Demand Analyst → Planner → Executor (subgraph) → Report Writer → Quality Reviewer → (loop back or finish) → Structure Organizer

Key insights:

- Planner decides next chapter, generates search queries
- Executor fans out queries in parallel (dynamic map)
- Writer synthesizes facts into prose with citations
- Reviewer routes back to Writer (fix) or Planner (next chapter)
- Keyword-based routing: `ROUTE: Planner` vs `ROUTE: Report Writer`

Pattern: Incremental refinement with quality gate loop

2. ReAct Agent

Flow: Task Normalizer → ReAct Subgraph (loop) → Final QA Editor

Key insights:

- Normalizer structures user intent as JSON
- Subgraph handles Thought→Action→Observation loop internally
- QA Editor checks if output meets constraints
- If "TODO" found in output, routes back to subgraph

Pattern: Preprocessing + iterative reasoning + post-validation

3. Loop Counter (Fixed Iterations)

Flow: Writer ↔ Critic ↔ Loop Gate → Finalizer

Key insights:

- Simple revision loop with hard cutoff
- Loop Gate counts entries, emits only on Nth pass
- No LLM needed for loop control

Pattern: Bounded iteration for revision tasks

4. Dynamic Fan-Out/Tree Aggregation

Flow: Multiple start nodes → Passthrough → Agent (map) → Agent (tree) → END

Key insights:

- Multiple `start` nodes fire in parallel
- Passthrough collects all inputs
- Map node processes each message in parallel
- Tree node aggregates results hierarchically (groups of N)

Pattern: Scatter-gather with hierarchical reduce

5. MACNet (DAG with Shared Context)

Flow: USER → Node(-1) → Node(0) → [Node(1), Node(2), Node(3)] → Node(-2) → END

Key insights:

- Diamond topology: one-to-many then many-to-one
- Each node is a subgraph (can be arbitrarily complex)
- Non-trigger edges (`trigger: false`) provide context without activation
- Process functions can run code on edge transitions

Pattern: Complex DAG with fan-out/fan-in and code execution

6. Reflexion Loop (Self-Critique)

Flow: Intake → Research → Reflexion Subgraph → Client QA

Key insights:

- Reflexion subgraph handles actor/critic loop internally
- Outer graph handles input normalization and output formatting
- Context flows forward to final editor for reference

Pattern: Nested self-improvement loop

Implementation Notes

Context Management

- `context_window: -1` = keep all messages (expensive)
- `context_window: 0` = stateless (cheapest)
- `context_window: N` = rolling window of N messages

Tool Integration

- Tools defined in `functions/` directory as Python
- Group syntax: `tool_group:All` or `tool_group:specific_tool`
- MCP support via `demo_mcp.yaml`

Execution

```
# Backend
uv run python server_main.py --port 6400

# Frontend
cd frontend && VITE_API_BASE_URL=http://localhost:6400 npm run dev

# SDK (programmatic)
```

```
from runtime.sdk import run_workflow
result = run_workflow(yaml_file="...", task_prompt="...", attachments=[])
```

Cost Considerations

Multi-agent = more API calls. Mitigations:

1. Use `context_window: 0` for stateless nodes
2. Use smaller models for routing/normalization (gpt-4o-mini)
3. Limit `max_parallel` to avoid burst costs
4. Use `loop_counter` to bound iteration loops
5. Consider their "puppeteer" approach (RL-based dynamic agent selection) for production

---

See Also

- `yaml_instance/` - all example workflows
- `docs/user_guide/en/workflow_authoring.md` - official docs
- MacNet paper: https://arxiv.org/abs/2406.07155
- Puppeteer paper: https://arxiv.org/abs/2505.19591